

第3章 基于MPI的分布式内存编程

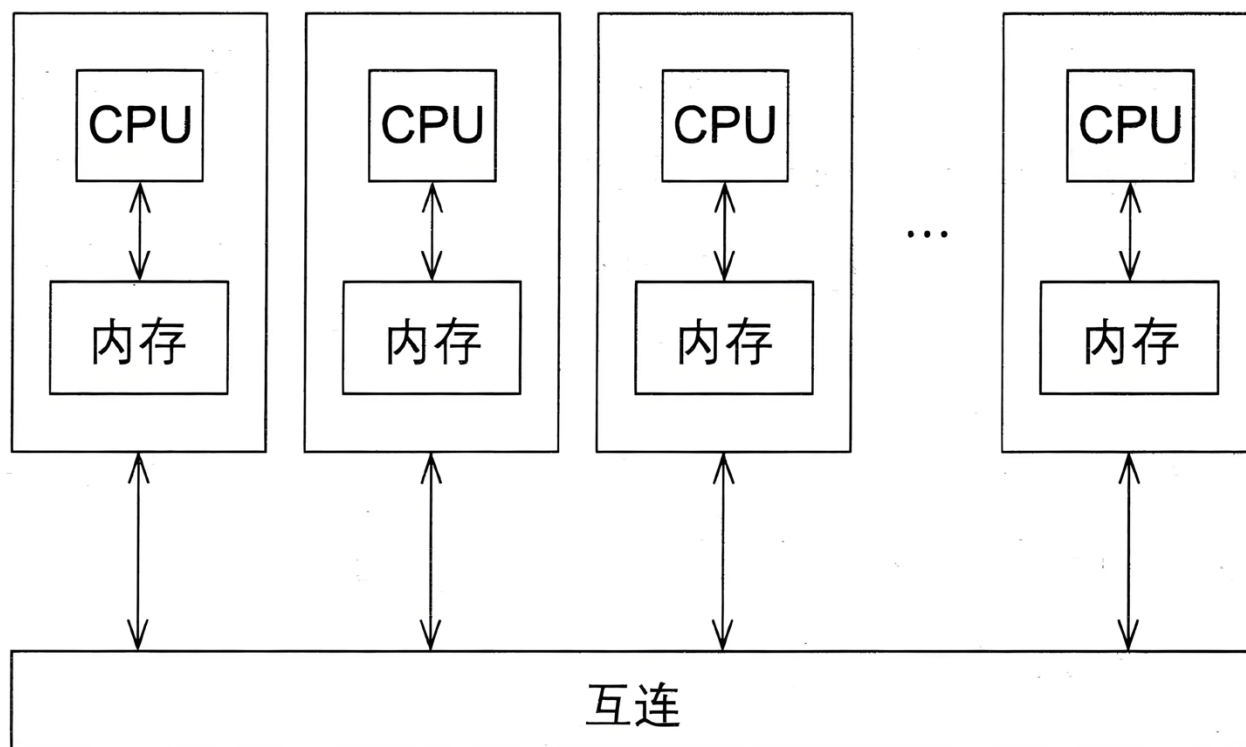
深圳大学计算机与软件学院 陆克中

路线图

- 编写你的第一个MPI程序
- 常用MPI函数的使用
- MPI派生数据类型
- 基于MPI的梯形积分法
- 集合通信
- 矩阵乘法
- MPI程序的性能评估
- MPI程序的安全性
- 线性方程组的求解
- 并行排序

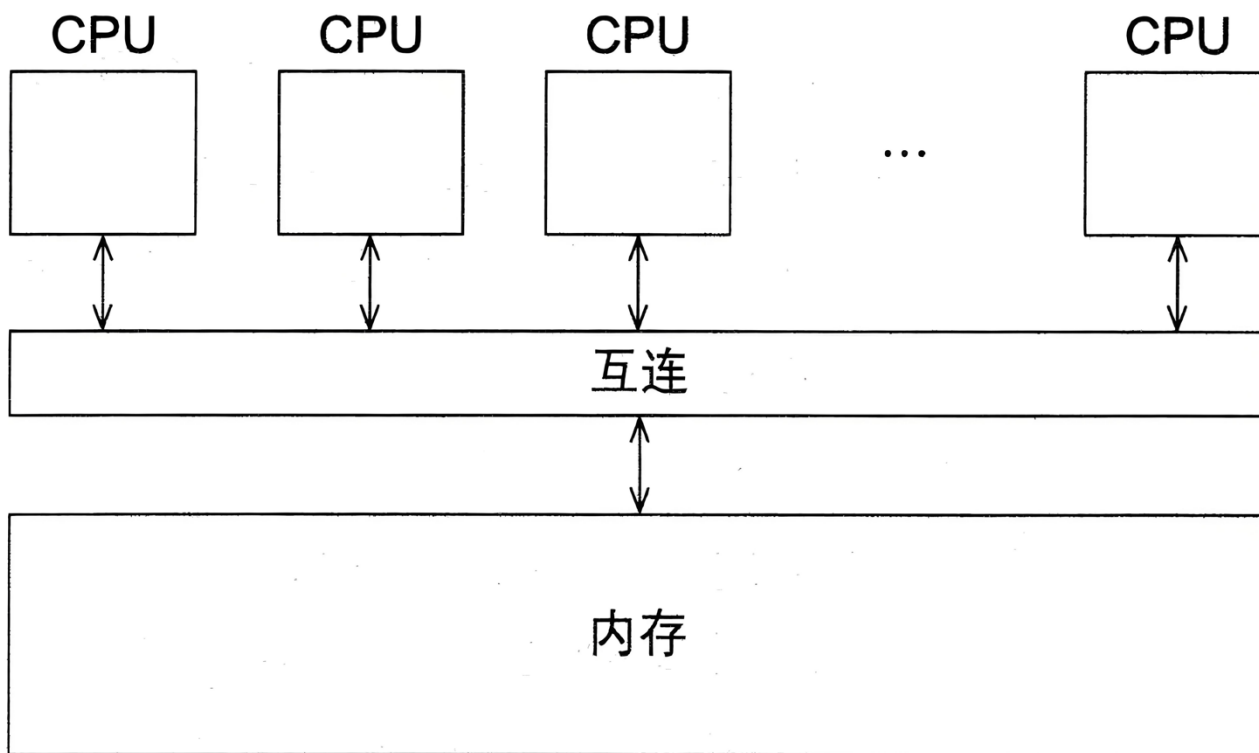
分布式内存系统

- 分布式内存系统由一些通过网络连接的核心 - 存储对组成，并且与核心相关的存储器只有该核心可以直接访问。



共享内存系统

- 共享内存系统由一组核心组成， 它们都连接了全局可访问的存储器。



Hello World!

- Kernighan和Ritchie经典的 “hello, world” 程序。

```
#include <stdio.h>

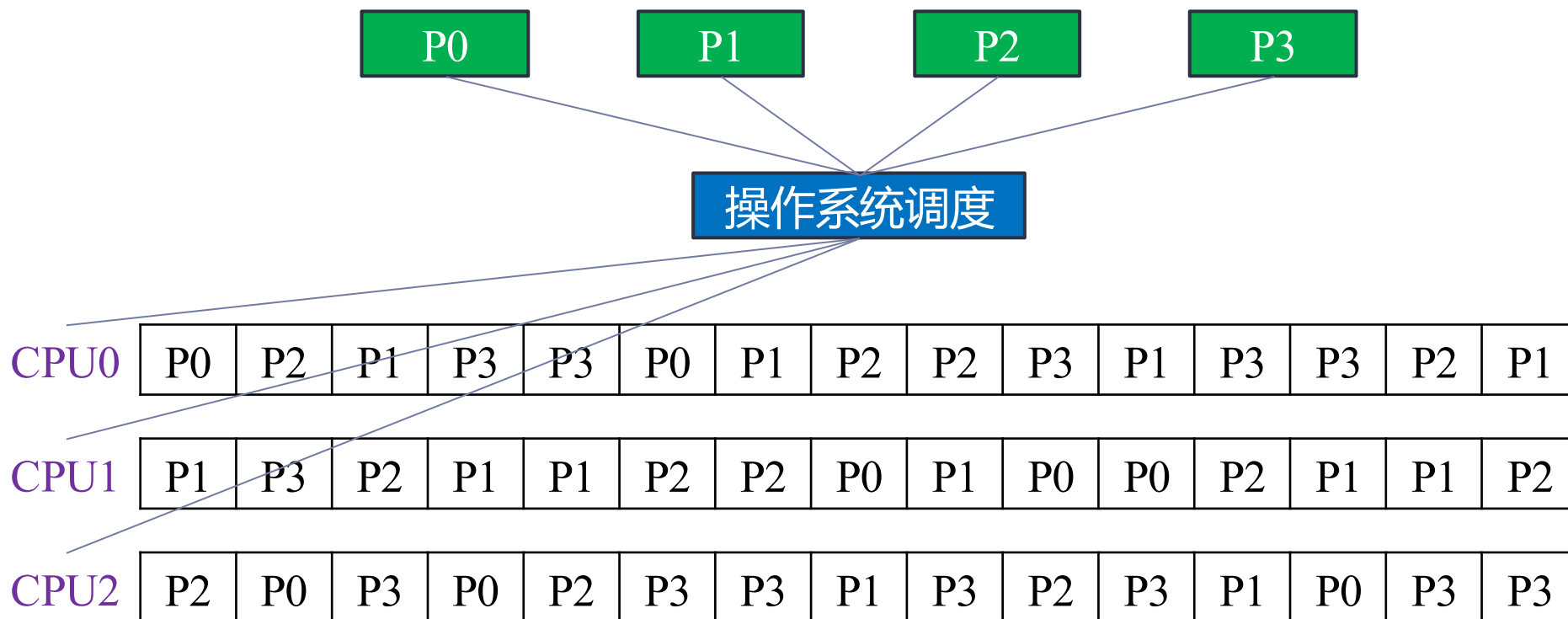
int main(void) {
    printf("hello, world\n");

    return 0;
}
```



识别MPI进程

- 用非负整数的序列号来表示进程。
- p 个进程的序列号为 $0, 1, 2, \dots, p-1$ 。



我们的第一个MPI程序

```
#include <stdio.h>
#include <string.h> //为了使用strlen
#include <mpi.h> //为了使用MPI函数等
const int MAX_STRING = 100;
int main(void) {
    char greeting[MAX_STRING];
    int size; //进程的数量
    int rank; //我的进程序列号
    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank != 0) {
        sprintf_s(greeting, "Greetings from process %d of %d!", rank, size);
        MPI_Send(greeting, strlen(greeting) + 1,
            MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    }
```

```
}
else {
    printf("Greetings from process %d of %d!\n", rank,
        size);
    for (int q = 1; q < size; q++) {
        MPI_Recv(greeting, MAX_STRING, MPI_CHAR,
            q, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
        printf("%s\n", greeting);
    }
}
MPI_Finalize();
return 0;
}
```



编译

■ 安装MPICH

- ▶ <https://www.mpich.org/downloads/>

■ Windows

- ▶ Visual Studio项目->属性
 - C/C++->附加包含目录: C:\Program Files (x86)\Microsoft SDKs\MPI\Include
 - 链接器->附加库目录: C:\Program Files (x86)\Microsoft SDKs\MPI\Lib\x64
 - 链接器->输入->附加依赖项: msmpi.lib

■ Linux

- ▶ C: `mpicc -O3 -o mpi.o mpi.c`
- ▶ C++: `mpic++ [-O3] [-g] [-Wall] -o mpi.o mpi.cpp`

编译器优化

生成调试信息

开启编译器警告

运行

■ Windows

- ▶ Windows PowerShell
- ▶ `mpiexec -n 4 "C:\Program Files\Microsoft MPI\Benchmarks\IMB-NBC.exe"`

■ Linux

- ▶ 查看集群状况: `pssh -h /etc/host_list echo date`
- ▶ `mpiexec -n 2 -f machinefile /home/bxjs/mpi.o`
- ▶ `mpiexec -n 10 -f machinefile /root/mpich-4.1/examples/cpi`

节点列表文件:



#格式: 主机名/IP地址 [: 进程数]

#注释以#开头, 进程数省略时默认1个进程

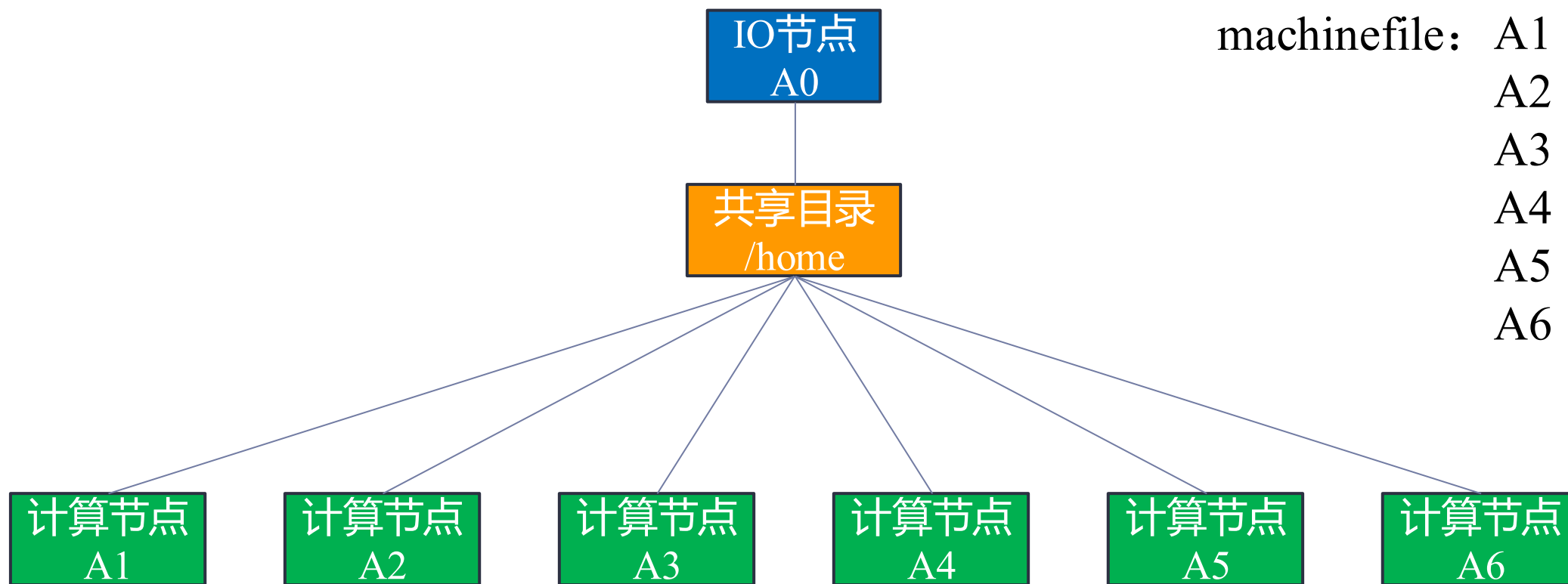
node01 #在node01节点启动1个进程

node02:4 #在node02节点启动4个进程

192.168.1.100:2 #在 IP 为192.168.1.100的节点启动2个进程

localhost:1 #在本地节点启动1个进程

集群结构



MPI程序

- 用C语言编写。
 - ▶ 包含main函数。
 - ▶ 使用stdio.h、 string.h等头文件。
- 需要添加mpi.h头文件。
- 由MPI定义的标识符均以 “MPI_” 开头。
- 下划线后的第一个字母为大写形式。
 - ▶ 此规则适用于函数名和MPI定义的类型。
 - ▶ 该命名规则有助于避免混淆。

MPI函数

■ MPI_Init

- ▶ 告诉MPI系统进行所有必要的设置。

```
int MPI_Init(  
    int*      argc_p    /* in/out */,  
    char***   argv_p    /* in/out */);
```

■ MPI_Finalize

- ▶ 告诉MPI系统已经完成MPI的使用，所有为MPI分配的资源都可以被释放。

```
int MPI_Finalize(void);
```

基本轮廓

```
. . .
#include <mpi.h>

. . .
int main(int argc, char* argv[]) {
    . . .
    /* 之前没有MPI调用 */
    MPI_Init(&argc, &argv);
    . . .
    MPI_Finalize();
    /* 之后没有MPI调用 */
    . . .
    return 0;
}
```

通信域



- 一组互相传递消息的进程的集合。
- MPI_Init函数会定义一个通信域，该通信域包含程序启动时创建的所有进程。
- 被称为**MPI_COMM_WORLD**。

```
int MPI_Comm_size(  
    MPI_Comm comm           /* in */,  
    int* comm_sz_p         /* out */);
```

通信域中进程的数量

```
int MPI_Comm_rank(  
    MPI_Comm comm           /* in */,  
    int* my_rank_p         /* out */);
```

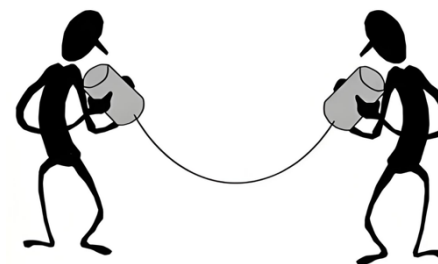
通信域中调用进程的序列号

SPMD

- 单程序多数据。
- 只编译一个程序。
- 进程0做的事情与其他进程完全不同：
 - 它接收一系列消息并打印，而其他每个进程都在创建和发送消息。
- **if-else**语句使我们的程序成为SPMD。

通信

```
int MPI_Send(  
    void*          msg_buf_p      /* in */,  
    int           msg_size       /* in */,  
    MPI_Datatype   msg_type       /* in */,  
    int           dest           /* in */,  
    int           tag            /* in */,  
    MPI_Comm       communicator  /* in */);
```

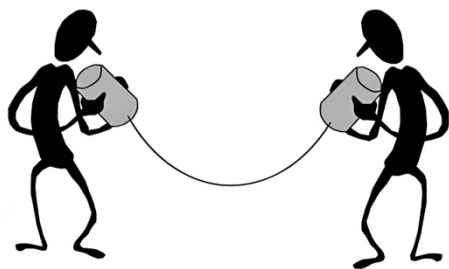


数据类型

MPI 数据类型	C 语言数据类型
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_LONG_LONG	signed long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

通信

```
int MPI_Recv(  
    void*          msg_buf_p      /* out */,  
    int           buf_size       /* in */,  
    MPI_Datatype   buf_type       /* in */,  
    int           source          /* in */,  
    int           tag             /* in */,  
    MPI_Comm       communicator   /* in */,  
    MPI_Status*    status_p       /* out */);
```



消息匹配

```
MPI_Send(send_buf_p, send_buf_sz, send_type, dest, send_tag,  
send_comm);
```

r

MPI_Send

src = q



MPI_Recv

dest = r

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,  
recv_comm, &status);
```

q

接收消息

■ 接收方可以在不知道以下信息的情况下接收消息：

- ▶ 消息中的数据量。
- ▶ 消息的发送方。
- ▶ 消息的标签。

■ $\text{send_buf_sz} \leq \text{recv_buf_sz}$

■ 通配符

- ▶ `MPI_ANY_SOURCE`
- ▶ `MPI_ANY_TAG`



```
#include "mpi.h"
#include <iostream>
using namespace std;
int main(void) {
    int rank, size;
    MPI_Status status;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (rank == 0)
        for (int i = 1; i < size; i++)
        {
            int a[20];
            MPI_Recv(a, 20, MPI_INT, i, i, MPI_COMM_WORLD, &status);
            int count;
            MPI_Get_count(&status, MPI_INT, &count);
            cout << "source = " << status.MPI_SOURCE << endl;
            cout << "tag = " << status.MPI_TAG << endl;
            cout << "count = " << count << endl;
            for (int i = 0; i < count; i++)
                cout << a[i] << " ";
            cout << endl << endl;
        }
    else {
        int a[20];
        for (int i = 0; i < 20; i++)
            a[i] = i;
        MPI_Send(a, 10 + rank, MPI_INT, 0, rank, MPI_COMM_WORLD);
    }
    MPI_Finalize();
    return 0;
}
```

status_p参数

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,  
recv_comm, &status);
```

MPI_Status*

MPI_SOURCE
MPI_TAG
MPI_ERROR

MPI_Status* status;

status.MPI_SOURCE

status.MPI_TAG

我接收到了多少数据？

```
int MPI_Get_count(  
    MPI_Status* status_p    /* in */,  
    MPI_Datatype type        /* in */,  
    int* count_p             /* out */);
```



MPI派生的数据类型



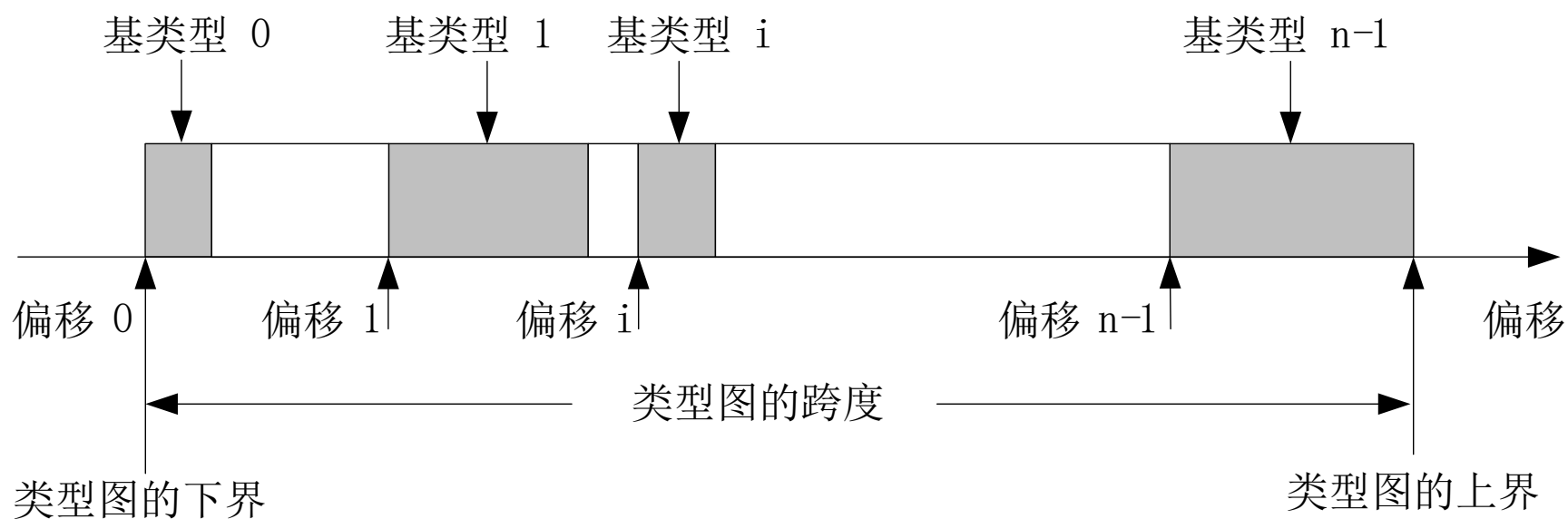
派生数据类型

- 通信比本地计算要昂贵得多。
- 在多个消息中发送固定数量数据的成本通常要比发送具有相同数据量的单个消息的成本大得多。

```
double x[1000];  
.  
.  
.  
if (my_rank == 0)  
    for (i = 0; i < 1000; i++)  
        MPI_Send(&x[i], 1, MPI_DOUBLE, 1, 0, comm);  
else /* my_rank == 1 */  
    for (i = 0; i < 1000; i++)  
        MPI_Recv(&x[i], 1, MPI_DOUBLE, 0, 0, comm, &status);  
  
if (my_rank == 0)  
    MPI_Send(x, 1000, MPI_DOUBLE, 1, 0, comm);  
else /* my_rank == 1 */  
    MPI_Recv(x, 1000, MPI_DOUBLE, 0, 0, comm, &status);
```


派生数据类型

- 定义由数据类型不同且地址空间不连续的数据项组成的消息。
- 可以用类型图来描述：基类型可以是任何MPI预定义数据类型，也可以是其他的派生数据类型。



派生数据类型

- 如果发送数据的函数知道数据项集合的类型和内存中的相对位置，它就可以在发送数据项之前从内存中收集这些数据项。
- 接收数据的函数可以在接收到数据项时将它们分发到内存中正确的目的地。

派生数据类型构造函数

函数名	含义
MPI_Type_contiguous	定义由相同数据类型的元素组成的类型。
MPI_Type_vector	定义由成块的元素组成的类型，块之间具有相同间隔。
MPI_Type_indexed	定义由成块的元素组成的类型，块长度和偏移由参数指定。
MPI_Type_struct	定义由不同数据类型的元素组成的类型。
MPI_Type_commit	提交一个派生数据类型。
MPI_Type_free	释放一个派生数据类型。

发送数组的所有偶序数元素

- MPI_Type_vector调用产生了派生数据类型EvenElements。
- MPI_Type_commit调用确认该派生数据类型。
- EvenElements的一个元素就包含A的所有50个偶序数元素。

```
double A[100];  
MPI_Datatype EvenElements;  
...  
MPI_Type_vector(50, 1, 2, MPI_DOUBLE, &EvenElements);  
MPI_Type_commit(&EvenElements);  
MPI_Send(A, 1, EvenElements, destination, ...);
```

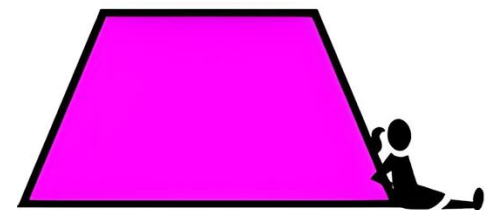
发送矩阵数据

- 矩阵
- 矩阵1行->矩阵1行
- 矩阵1列->矩阵1列
- 矩阵2列->矩阵2列
- 矩阵1列->矩阵1行
- 矩阵1行->矩阵1列
- 矩阵对角线

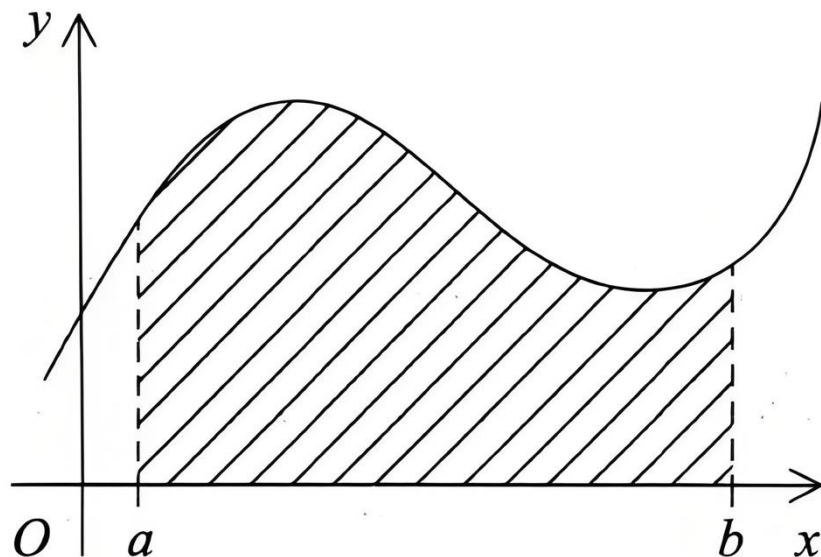
```
#include "mpi.h"
#include <iostream>
#define N 10
using namespace std;
int main(int argc, char* argv[])
{
    int rank;
    int a[N][N];
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Datatype Column_Type;
    MPI_Type_vector(N, 1, N, MPI_INT,
        &Column_Type);
    MPI_Type_commit(&Column_Type);
    if (rank == 0)
    {
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                a[i][j] = i * N + j;
        MPI_Send(a[0], 1, Column_Type, 1, 0,
```

```
MPI_COMM_WORLD);
    }
    else
    {
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                a[i][j] = -1;
        MPI_Status status;
        MPI_Recv(a[0], 1, Column_Type, 0, 0,
            MPI_COMM_WORLD, &status);
        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
                cout << a[i][j] << "\t";
            cout << endl;
        }
        MPI_Finalize();
        return 0;
    }
}
```

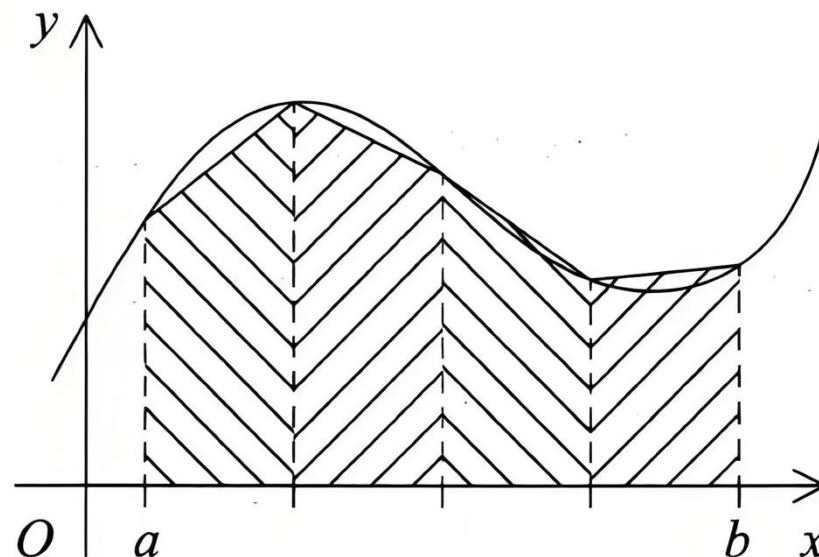
MPI中的梯形法则



梯形法则



拟估算面积



利用梯形近似面积

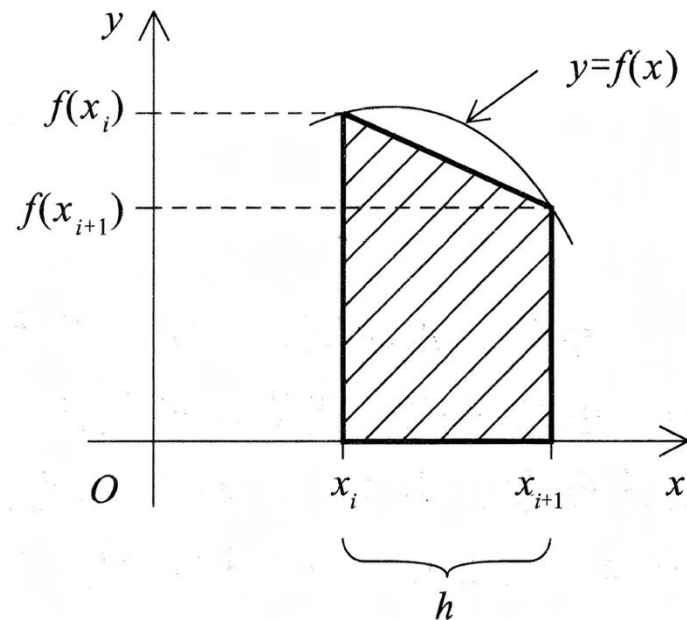
梯形法则

$$\text{梯形面积} = \frac{h}{2} [f(x_i) + f(x_{i+1})]$$

$$h = \frac{b-a}{n}$$

$$x_0 = a, x_1 = a + h, x_2 = a + 2h, \dots, x_{n-1} = a + (n-1)h, x_n = b$$

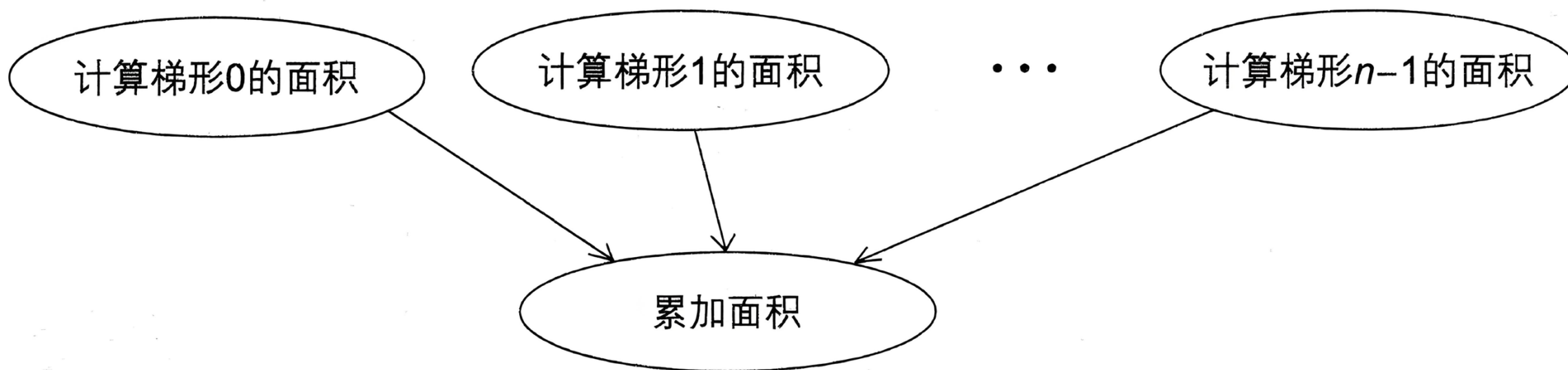
$$\text{梯形面积的总和} = h[f(x_0)/2 + f(x_1) + f(x_2) + \dots + f(x_{n-1}) + f(x_n)/2]$$



串行程序的伪代码

```
/* 输入:  a, b, n */  
h = (b-a)/n;  
approx = (f(a) + f(b))/2.0;  
for (i = 1; i <= n-1; i++) {  
    x_i = a + i*h;  
    approx += f(x_i);  
}  
approx = h*approx;
```

梯形法则的并行化



梯形法则的任务和通信

并行伪代码

```
1      Get a, b, n;
2      h = (b-a)/n;
3      local_n = n/comm_sz;
4      local_a = a + my_rank*local_n*h;
5      local_b = local_a + local_n*h;
6      local_integral = Trap(local_a, local_b, local_n, h);
7      if (my_rank != 0)
8          Send local_integral to process 0;
9      else /* my_rank == 0 */
10         total_integral = local_integral;
11         for (proc = 1; proc < comm_sz; proc++) {
12             Receive local_integral from proc;
13             total_integral += local_integral;
14         }
15     }
16     if (my_rank == 0)
17         print result;
```

并行程序

```
#include <stdio.h>
#include <mpi.h>
double f(double x) {
    return x * x; //4 / (1 + x * x);
}
double Trap(double left_endpt, double
right_endpt, int trap_count, double
base_len) {
    double estimate, x;
    int i;
    estimate = (f(left_endpt) + f(right_endpt))
/ 2.0;
    for (i = 1; i <= trap_count - 1; i++) {
        x = left_endpt + i * base_len;
        estimate += f(x);
    }
    estimate = estimate * base_len;
    return estimate;
}
int main(void) {
    int rank, size, n, local_n;
```

```
double a = 0.0, b = 3.0, h, local_a,
local_b;
double local_int, total_int;
MPI_Init(NULL, NULL);
MPI_Comm_rank(MPI_COMM_WORL
D, &rank);
MPI_Comm_size(MPI_COMM_WORL
D, &size);
if (rank == 0)
    scanf_s("%d", &n);
MPI_Bcast(&n, 1, MPI_INT, 0,
MPI_COMM_WORLD);
h = (b - a) / n; //h对所有进程相同
local_n = n / size; //所以梯形的数量也
相同
local_a = a + rank * local_n * h;
local_b = local_a + local_n * h;
local_int = Trap(local_a, local_b, local_n,
h);
if (rank != 0) {
    MPI_Send(&local_int, 1, MPI_DOUBLE,
```

```
0, 0, MPI_COMM_WORLD);
}
else {
    total_int = local_int;
    for (int source = 1; source < size;
source++) {
        MPI_Recv(&local_int, 1, MPI_DOUBLE,
source, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        total_int += local_int;
    }
}
if (rank == 0) {
    printf("With n = %d trapezoids, our
estimate\n", n);
    printf("of the integral from %f to %f =
%.15e\n", a, b, total_int);
}
MPI_Finalize();
return 0;
}
```

输入

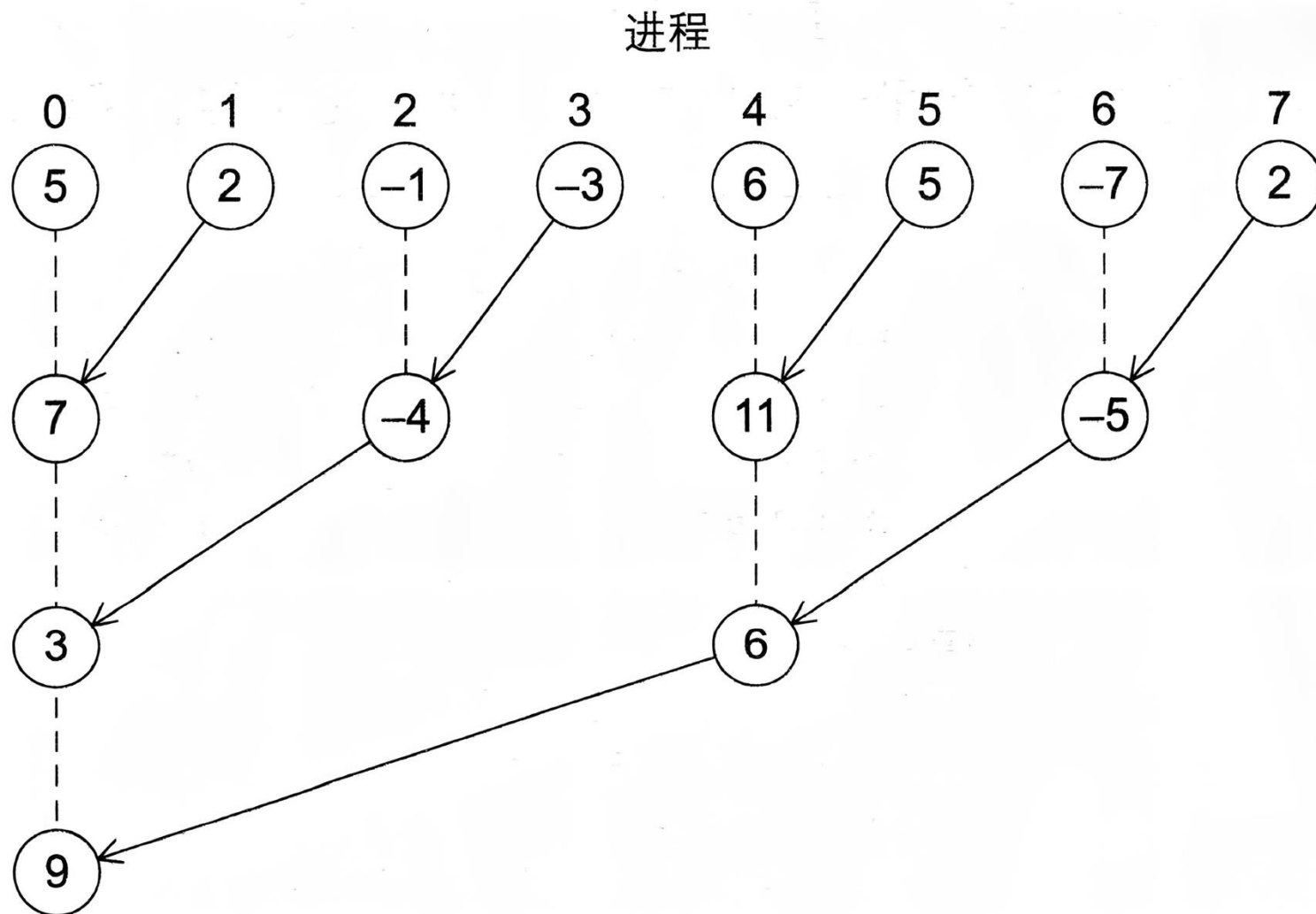
- 大多数MPI实现只允许MPI_COMM_WORLD 中的进程0访问stdin。
- 进程0必须读取数据（通过 scanf）并将其发送给其他进程。

```
if (rank == 0)
    scanf_s("%d", &n);
MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

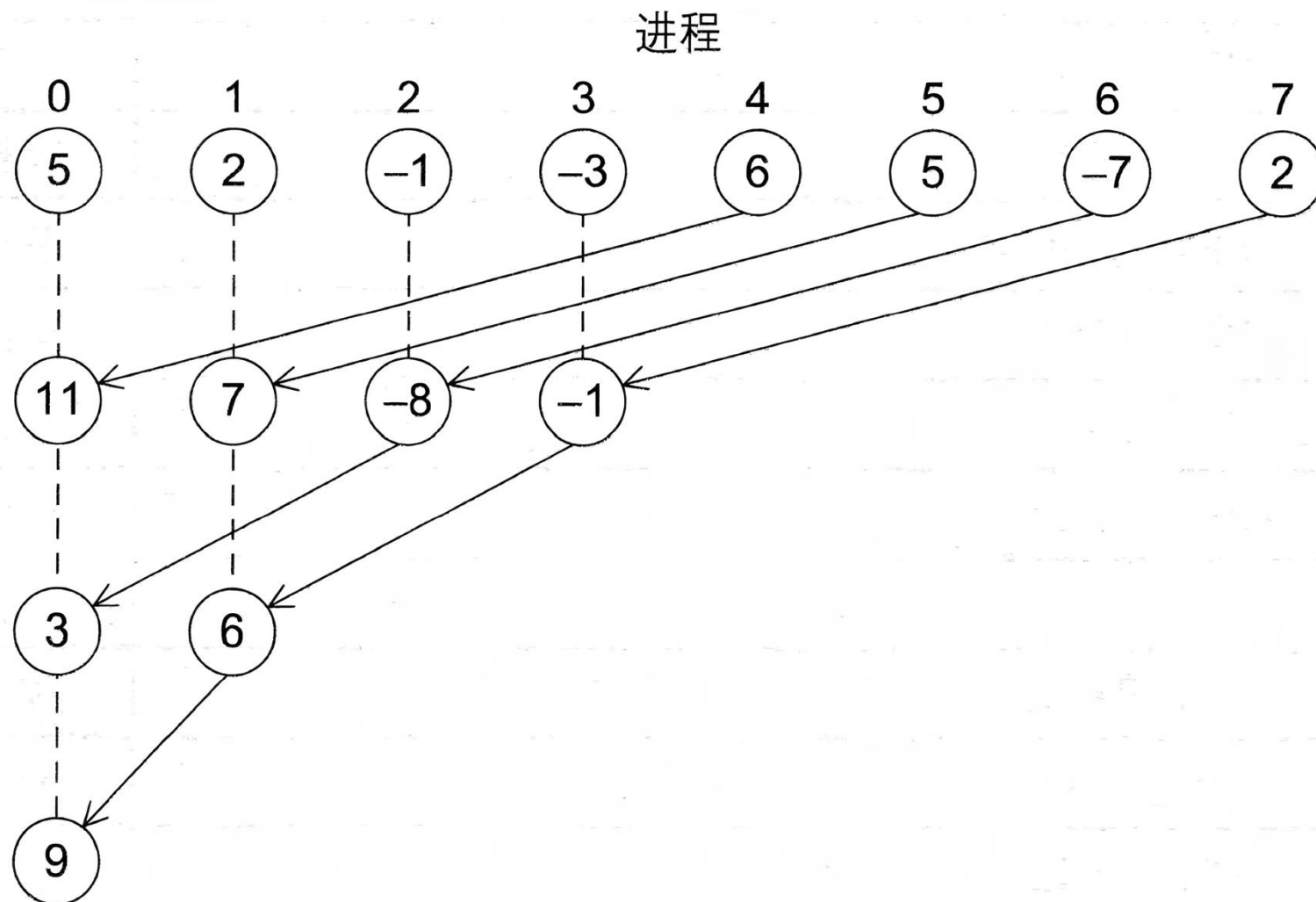
集合通信



树形结构的全局和



另一种树状结构的全局和



MPI_Reduce

```
#include <stdio.h>
#include <mpi.h>
double f(double x) {
    return x * x;
}
double Trap(double left_endpt,
double right_endpt, int trap_count,
double base_len) {
    double estimate, x;
    int i;
    estimate = (f(left_endpt) +
f(right_endpt)) / 2.0;
    for (i = 1; i <= trap_count - 1; i++) {
        x = left_endpt + i * base_len;
        estimate += f(x);
    }
    estimate = estimate * base_len;
    return estimate;
}
```

```
}
int main(void) {
    int rank, size, n = 1024, local_n;
    double a = 0.0, b = 3.0, h, local_a,
    local_b;
    double local_int, total_int;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_W
ORLD, &rank);
    MPI_Comm_size(MPI_COMM_W
ORLD, &size);
    MPI_Bcast(&n, 1, MPI_INT, 0,
MPI_COMM_WORLD);
    h = (b - a) / n; /* h对所有进程相同 */
    local_n = n / size; /* 所以梯形的数
量也相同 */
    local_a = a + rank * local_n * h;
```

```
local_b = local_a + local_n * h;
local_int = Trap(local_a, local_b,
local_n, h);
MPI_Reduce(&local_int, &total_int,
1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
if (rank == 0) {
    printf("With n = %d trapezoids, our
estimate\n", n);
    printf("of the integral from %f to %f
= %.15e\n", a, b, total_int);
}
MPI_Finalize();
return 0;
}
```

MPI_Reduce

```
int MPI_Reduce(  
    void*          input_data_p    /* in */,  
    void*          output_data_p  /* out */,  
    int            count           /* in */,  
    MPI_Datatype    datatype       /* in */,  
    MPI_Op          operator       /* in */,  
    int            dest_process    /* in */,  
    MPI_Comm        comm          /* in */);
```

```
MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM,  
          0, MPI_COMM_WORLD);
```

```
double local_x[N], sum[N];  
.  
.  
.  
MPI_Reduce(local_x, sum, N, MPI_DOUBLE, MPI_SUM, 0,  
          MPI_COMM_WORLD);
```

MPI中的预定义归约操作符

操作值	含义
MPI_MAX	最大
MPI_MIN	最小
MPI_SUM	总和
MPI_PROD	乘积
MPI LAND	逻辑与
MPI_BAND	按位与
MPI_LOR	逻辑或
MPI BOR	按位或
MPI_LXOR	逻辑异或
MPI_BXOR	按位异或
MPI_MAXLOC	最大值和最大值的位置
MPI_MINLOC	最小值和最小值的位置

集合通信与点对点通信

- 通信域中的**所有**进程必须调用相同的集合函数。
 - ▶ 例如，如果一个程序试图在一个进程上调用MPI_Reduce，而在另一个进程上调用MPI_Recv，那么这个程序就会出错，并且很可能会挂起或崩溃。
- 每个进程传递给MPI集合通信的参数必须是兼容的。
 - ▶ 例如，如果一个进程传入0作为dest_process，而另一个进程传入1，那么调用MPI_Reduce的结果就是错误的，程序可能再次挂起或崩溃。
- output_data_p参数只在dest_process上使用。但是，所有进程仍然需要传入与output_data_p对应的实际参数，即使它只是NULL。
- 点对点通信基于标签和通信域进行匹配。集合通信不使用标签，所以它们完全根据通信域和被调用的顺序来匹配。

MPI_Reduce的多次调用

- 乍一看，似乎在两次调用MPI_Reduce之后，b的值将是3，d的值将是6。
- 但是，内存位置的名称与调用MPI_Reduce的匹配无关。
- 调用的顺序将决定匹配，因此存储在b中的值将是 $1+2+1=4$ ，存储在d中的值将是 $2+1+2=5$ 。

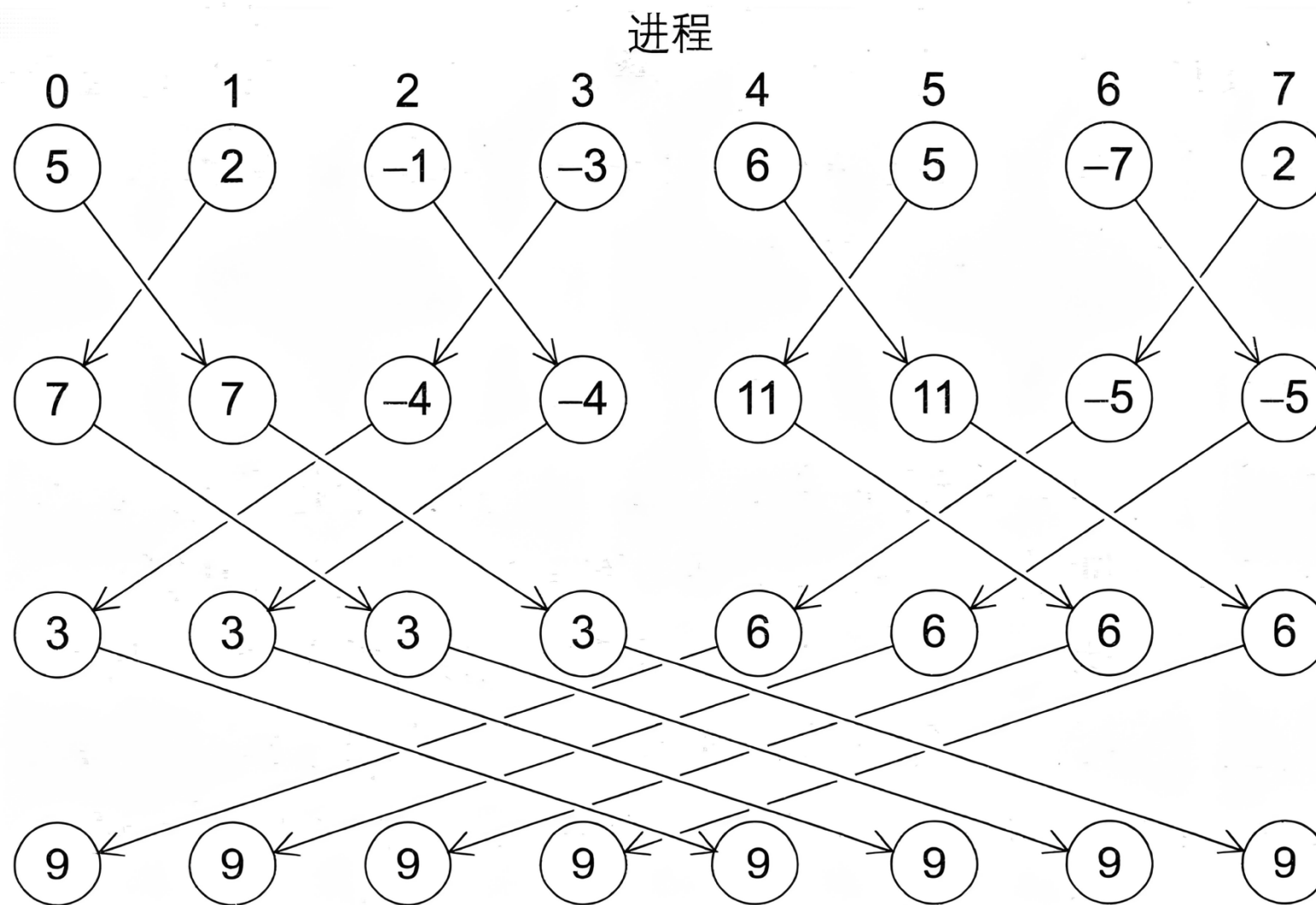
```
#include <stdio.h>
#include <mpi.h>
int main(void) {
    int rank, a, b, c, d;
    a = 1;
    c = 2;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0 || rank == 2)
    {
        MPI_Reduce(&a, &b, 1,
            MPI_INT, MPI_SUM, 0,
            MPI_COMM_WORLD);
        MPI_Reduce(&c, &d, 1,
            MPI_INT, MPI_SUM, 0,
            MPI_COMM_WORLD);
    }
    else {
```

```
        MPI_Reduce(&c, &d, 1,
            MPI_INT, MPI_SUM, 0,
            MPI_COMM_WORLD);
    }
    if (rank == 0)
        printf("b = %d, d = %d\n",
            b, d);
    MPI_Finalize();
    return 0;
}
```

MPI_Allreduce

- 所有进程都需要一个全局和的结果来完成一些更复杂的计算。

```
int MPI_Allreduce(  
    void*          input_data_p    /* in */,  
    void*          output_data_p  /* out */,  
    int            count           /* in */,  
    MPI_Datatype    datatype       /* in */,  
    MPI_Op          operator       /* in */,  
    MPI_Comm        comm           /* in */);
```

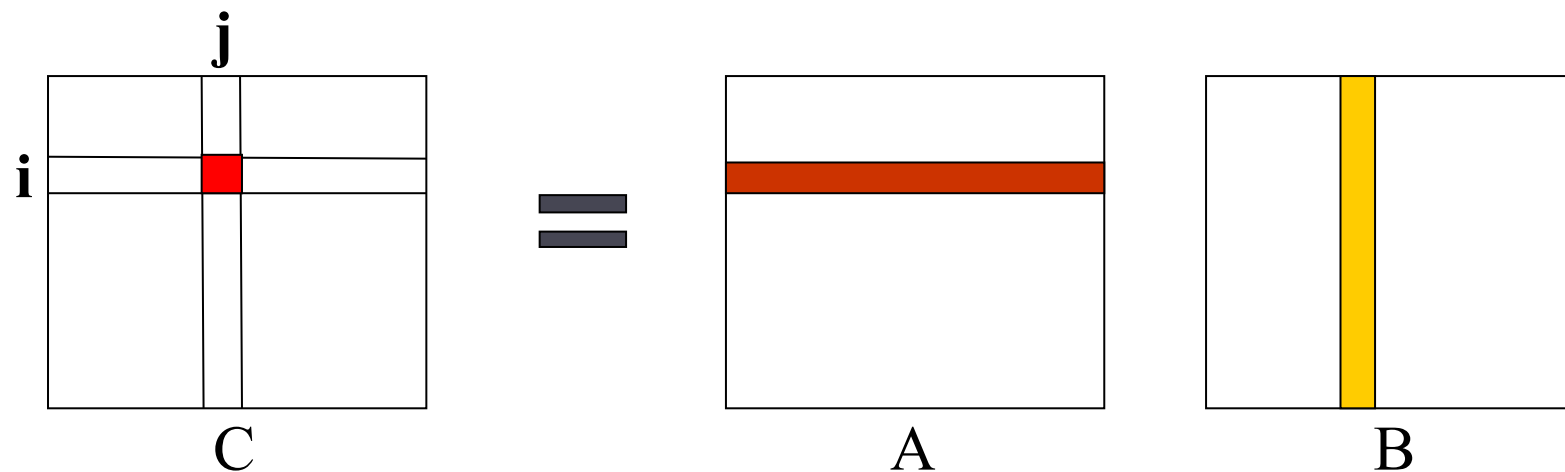
蝶式模式的全局和

矩阵乘法

矩阵乘法

设 $A = (a_{ij})_{n \times n}$ $B = (b_{ij})_{n \times n}$ $C = (c_{ij})_{n \times n}$, $C = A \times B$

$$\begin{pmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,n-1} \\ c_{1,0} & c_{1,1} & & c_{1,n-1} \\ \vdots & \vdots & & \vdots \\ c_{n-1,0} & c_{n-1,1} & \cdots & c_{n-1,n-1} \end{pmatrix} = \begin{pmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n-1} \\ a_{1,0} & a_{1,1} & & a_{1,n-1} \\ \vdots & \vdots & & \vdots \\ a_{n-1,0} & a_{n-1,1} & \cdots & a_{n-1,n-1} \end{pmatrix} \cdot \begin{pmatrix} b_{0,0} & b_{0,1} & \cdots & b_{0,n-1} \\ b_{1,0} & b_{1,1} & & b_{1,n-1} \\ \vdots & \vdots & & \vdots \\ b_{n-1,0} & b_{n-1,1} & \cdots & b_{n-1,n-1} \end{pmatrix}$$

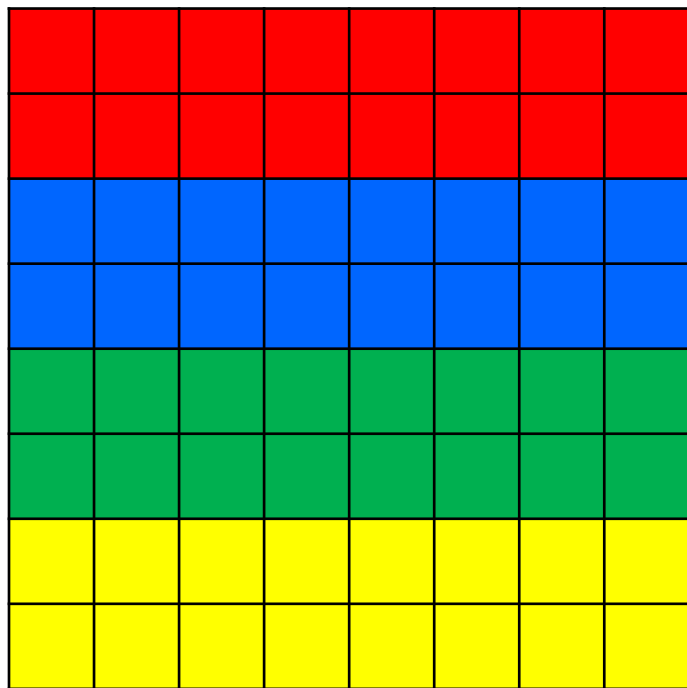


$$c_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$$

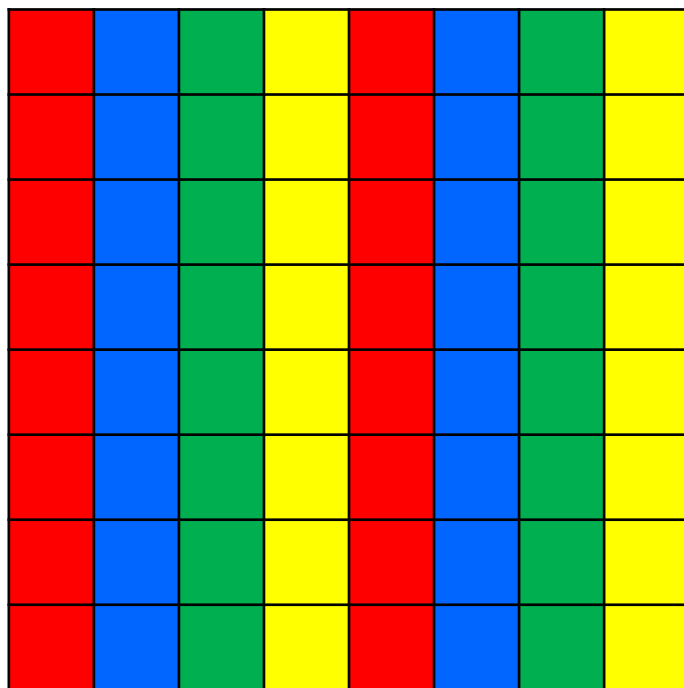
矩阵的划分

- 行、列、棋盘。
- 块、循环、块循环。

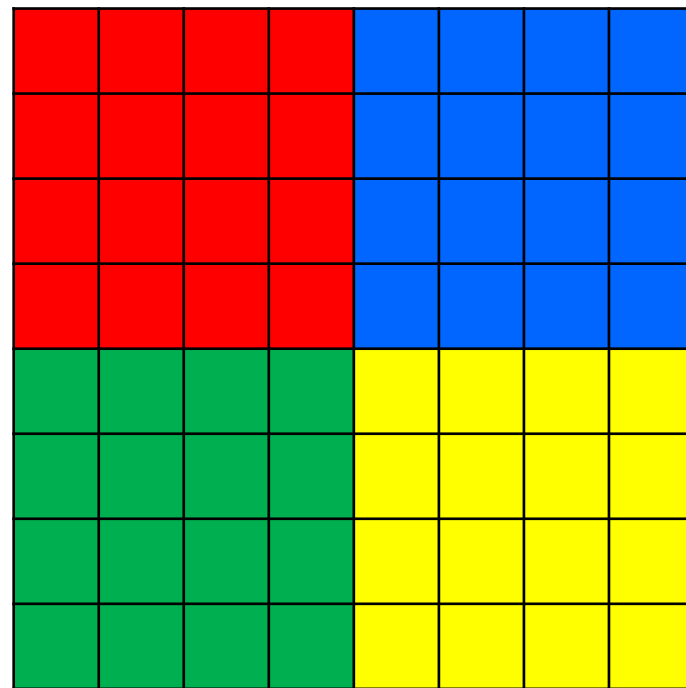
■ P0 ■ P1 ■ P2 ■ P3



块行划分



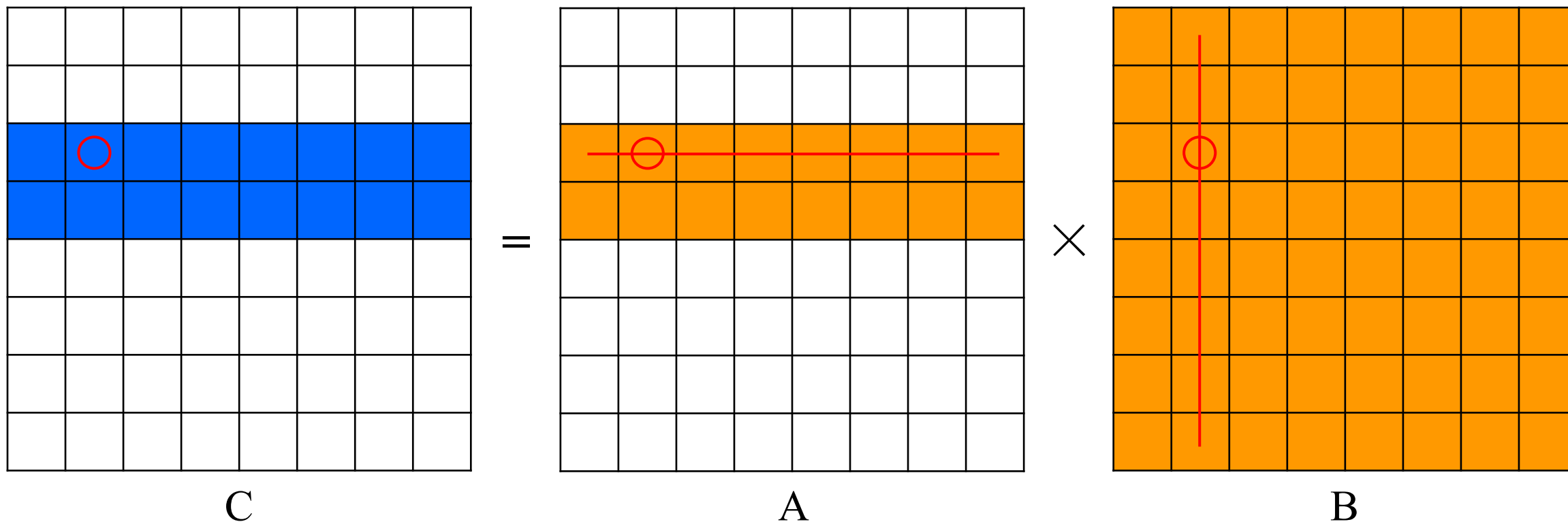
循环列划分



块棋盘划分

块行划分的矩阵乘法

- 计算 C_i , 需要 A_i 和 B , $0 \leq i < p$.



块行划分的矩阵乘法

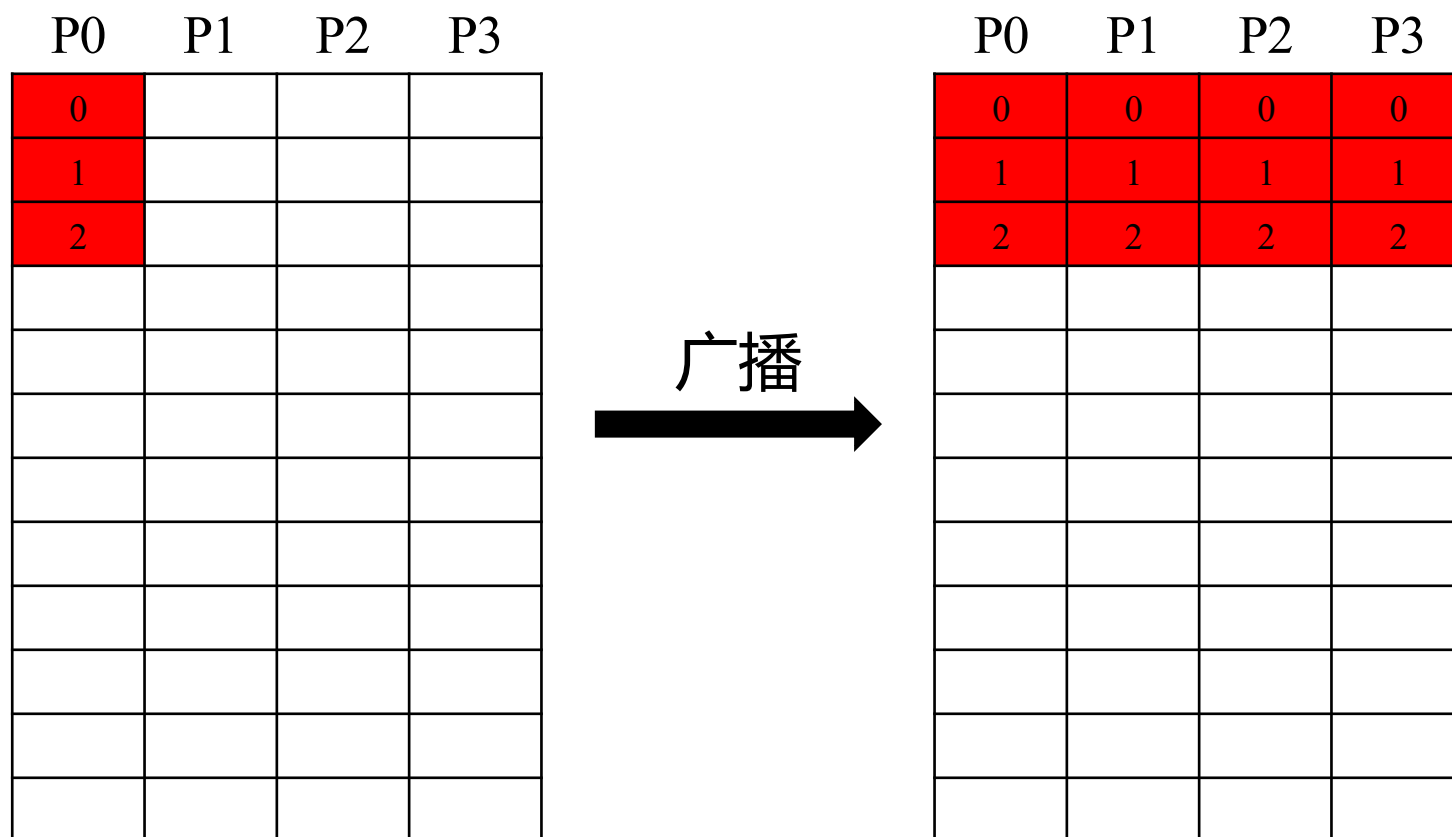
```
#include "mpi.h"
#include <iostream>
using namespace std;
int main(int argc, char* argv[])
{
    int n = 1024;
    double* a = NULL, * b = NULL, * c =
    NULL;
    double* local_a = NULL, * local_c =
    NULL;
    int rank, size, m;
    if (argc > 1)
        n = atoi(argv[1]);
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD,
    &rank);
    MPI_Comm_size(MPI_COMM_WORLD,
    &size);
    m = n / size;
    local_a = new double[m * n];
    b = new double[n * n];
    local_c = new double[m * n];
```

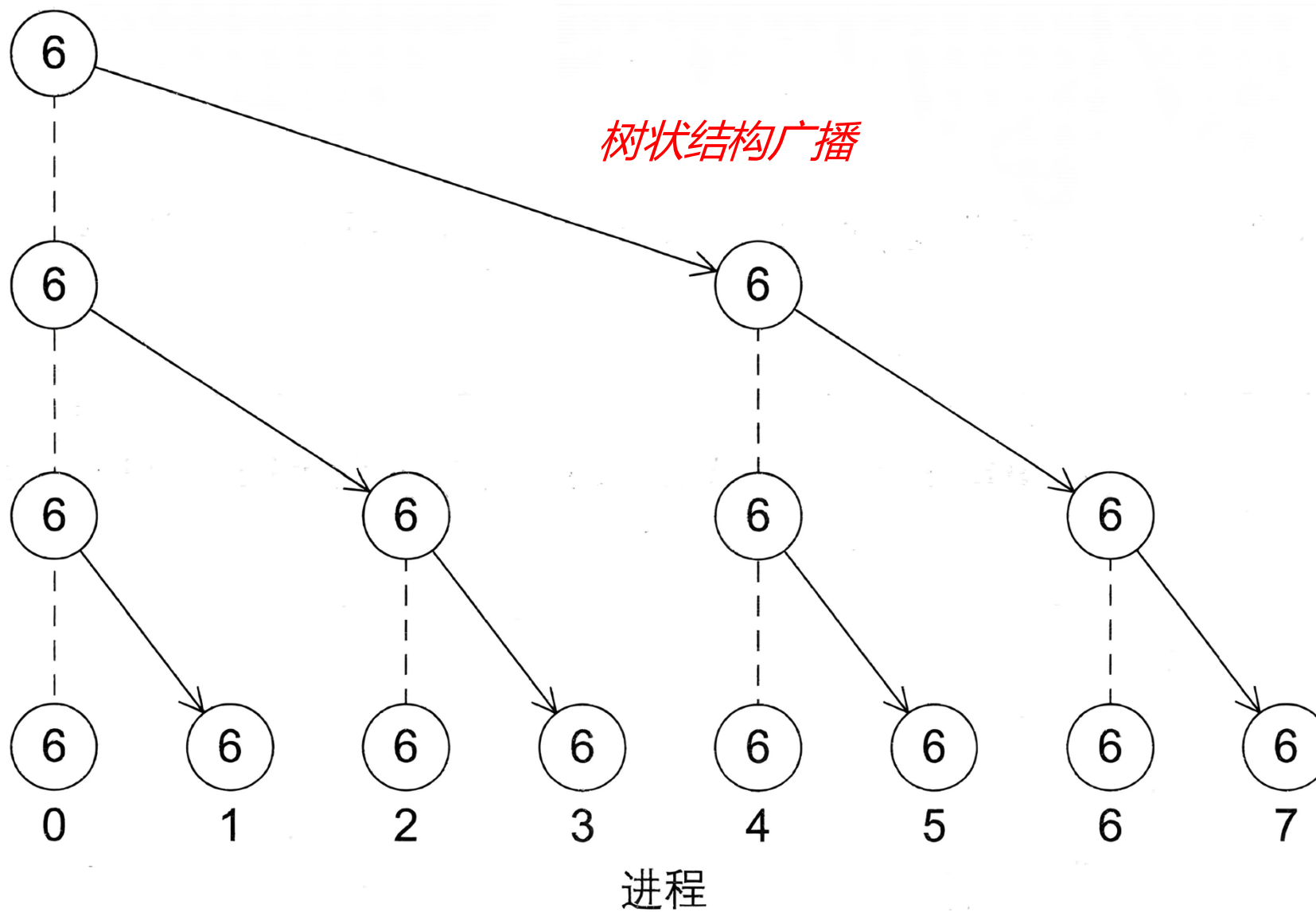
```
    if (rank == 0)
    {
        a = new double[n * n];
        c = new double[n * n];
        srand(0);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
            {
                a[i * n + j] = rand() * 1.0 / RAND_MAX;
                b[i * n + j] = rand() * 1.0 / RAND_MAX;
            }
    }
    double start = MPI_Wtime();
    MPI_Scatter(a, m * n, MPI_DOUBLE,
    local_a, m * n, MPI_DOUBLE, 0,
    MPI_COMM_WORLD);
    MPI_Bcast(b, n * n, MPI_DOUBLE, 0,
    MPI_COMM_WORLD);
    for (int i = 0; i < m * n; i++)
        local_c[i] = 0;
    for (int k = 0; k < n; k++)
        for (int i = 0; i < m; i++)
```

```
        for (int j = 0; j < n; j++)
            local_c[i * n + j] += local_a[i * n + k] * b[k
            * n + j];
    MPI_Gather(local_c, m * n,
    MPI_DOUBLE, c, m * n, MPI_DOUBLE,
    0, MPI_COMM_WORLD);
    double finish = MPI_Wtime();
    if (rank == 0)
        cout << "Elapsed time = " << finish - start
        << endl;
    MPI_Finalize();
    delete[] local_a;
    delete[] b;
    delete[] local_c;
    if (rank == 0)
    {
        delete[] a;
        delete[] c;
    }
    return 0;
}
```

MPI_Bcast

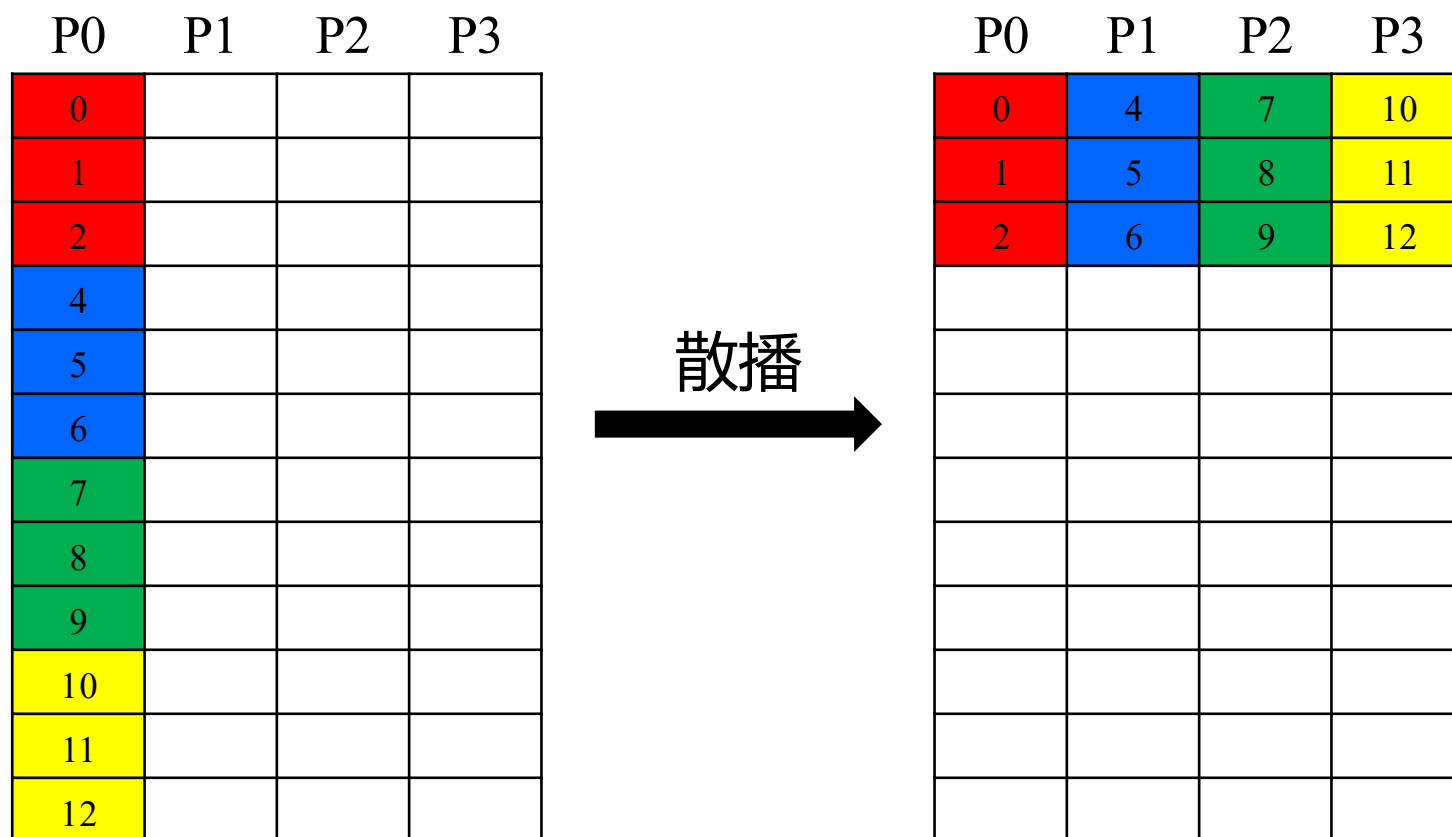
- Root进程发送相同的消息给通信域Comm中的所有进程。





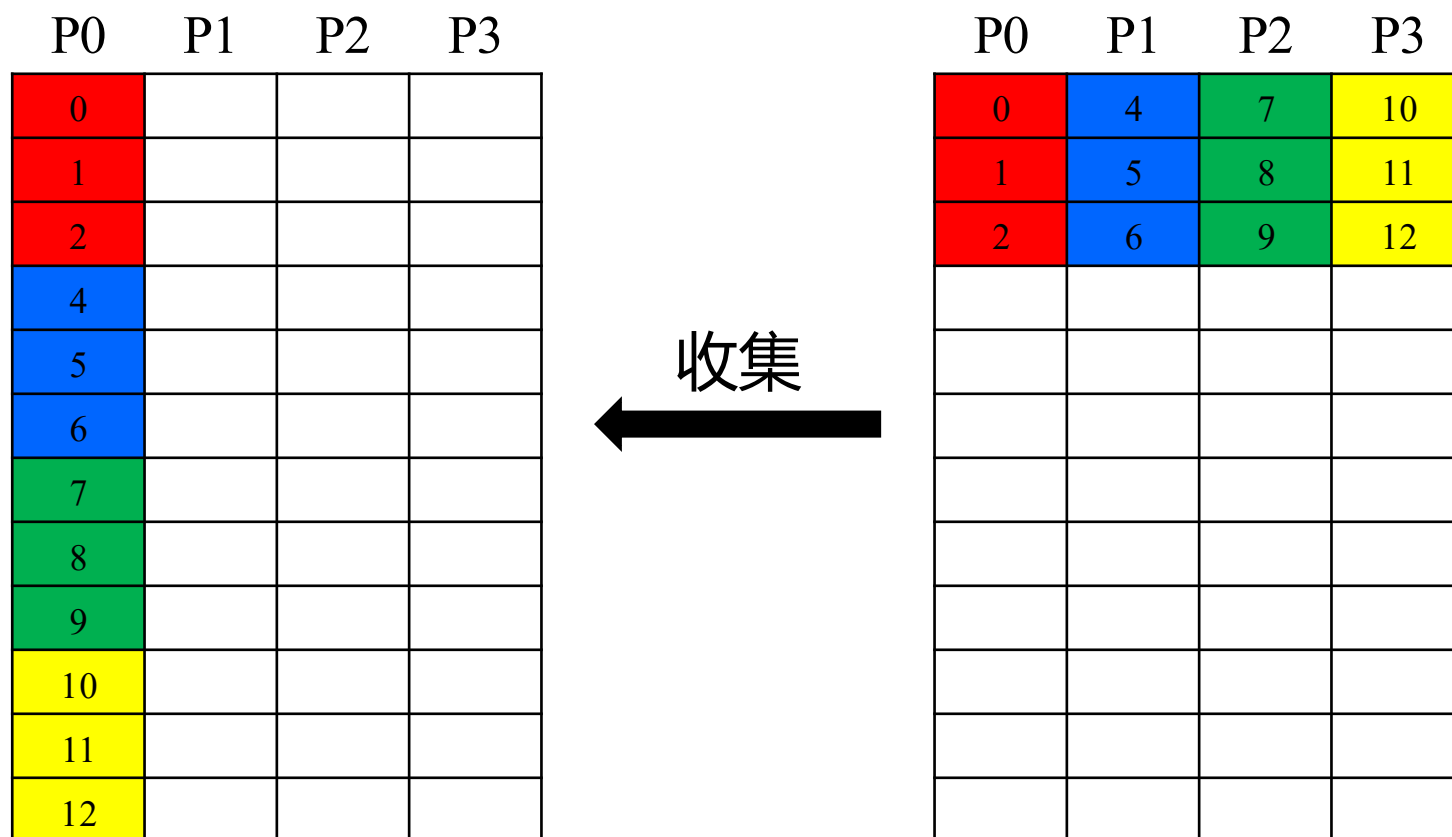
MPI_Scatter

- Root进程给所有进程发送一个不同的消息，这些消息按进程标识的顺序有序地存放在其发送缓冲区中。



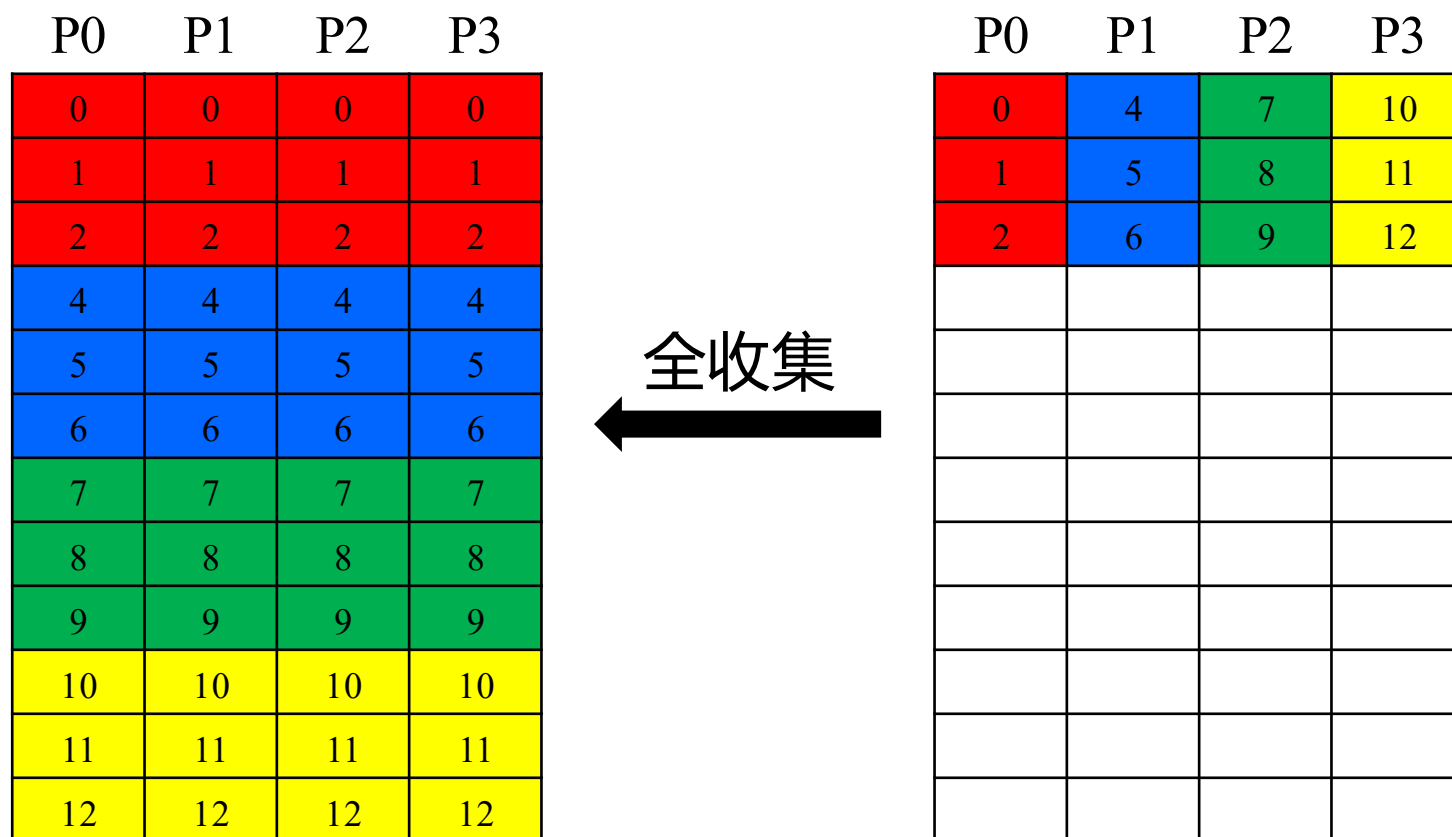
MPI_Gather

- Root进程从通信域Comm的所有进程接收消息，并按照进程的标识排序进行拼接，然后存放在其接收缓冲中。



MPI_Allgather

- 每一个进程都收集来自所有进程的数据。



性能评估



MPI_Wtime

- 返回从某个固定（但未指定）时间点开始的墙上时钟时间（wall-clock time），单位为秒，精度通常达到微秒甚至纳秒级别。

```
double start, finish;
```

```
. . .
```

```
start = MPI_Wtime();
```

```
/* 要计时的代码 */
```

```
. . .
```

```
finish = MPI_Wtime();
```

```
printf("Proc %d > Elapsed time = %e seconds\n"  
      my_rank, finish-start);
```



并行矩阵乘法的运行时间

矩阵阶 进程数	256	512	768	1024	1280	1536	1792	2048
1	0.0441485	0.345854	4.22938	9.87563	19.8494	35.0955	61.8041	128.49
2	0.031529	0.208856	2.20658	5.13127	10.2949	18.0625	31.4696	65.2809
4	0.0292768	0.151716	1.20752	2.75494	5.53627	9.59025	16.7371	34.2683
8	0.0407721	0.144538	0.738947	1.66798	3.45407	6.31026	9.58292	18.427
16	0.042161	0.151338	0.551712	1.29439	2.38452	4.56727	6.00456	10.6598
32	0.0565679	0.202048	0.566392	1.25515	2.08949	3.6645	4.64039	7.39174
64	0.0863599	0.308989	0.723337	1.39783	2.23502	3.51996	4.70449	6.82619

加速比和效率

$$S(n, p) = \frac{T_{\text{serial}}(n)}{T_{\text{parallel}}(n, p)}$$

$$E(n, p) = \frac{S(n, p)}{p} = \frac{T_{\text{serial}}(n)}{p \times T_{\text{parallel}}(n, p)}$$

并行矩阵乘法的加速比

矩阵阶 进程数	256	512	768	1024	1280	1536	1792	2048
1	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
2	1.40	1.66	1.92	1.92	1.93	1.94	1.96	1.97
4	1.51	2.28	3.50	3.58	3.59	3.66	3.69	3.75
8	1.08	2.39	5.72	5.92	5.75	5.56	6.45	6.97
16	1.05	2.29	7.67	7.63	8.32	7.68	10.29	12.05
32	0.78	1.71	7.47	7.87	9.50	9.58	13.32	17.38
64	0.51	1.12	5.85	7.06	8.88	9.97	13.14	18.82

并行矩阵乘法的效率

矩阵阶 进程数	256	512	768	1024	1280	1536	1792	2048
1	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
2	0.70	0.83	0.96	0.96	0.96	0.97	0.98	0.98
4	0.38	0.57	0.88	0.90	0.90	0.91	0.92	0.94
8	0.14	0.30	0.72	0.74	0.72	0.70	0.81	0.87
16	0.07	0.14	0.48	0.48	0.52	0.48	0.64	0.75
32	0.02	0.05	0.23	0.25	0.30	0.30	0.42	0.54
64	0.01	0.02	0.09	0.11	0.14	0.16	0.21	0.29

可扩展性

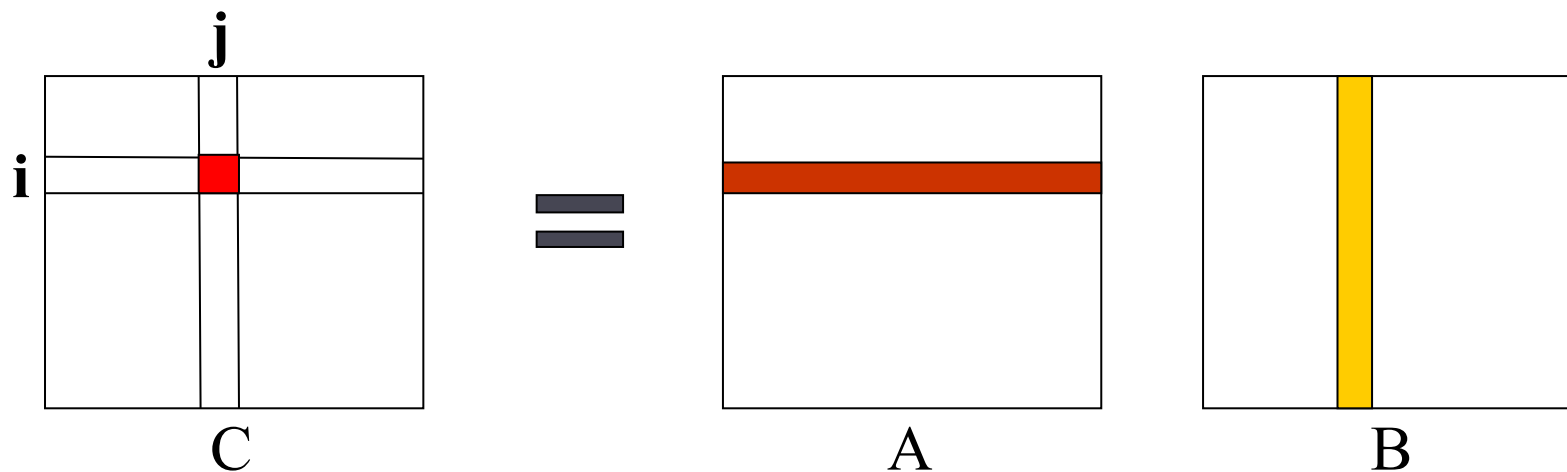
- 若程序的问题规模能够按一定速率扩大，使得效率不会随进程数量的增加而降低，则该程序具备**可扩展性**。
- 无需扩大问题规模就能保持效率恒定的程序，有时被称为具备**强可扩展性**。
- 若程序在问题规模与进程数量按相同速率扩大时能保持效率恒定，则该程序有时被称为具备**弱可扩展性**。



矩阵乘法

设 $A = (a_{ij})_{n \times n}$ $B = (b_{ij})_{n \times n}$ $C = (c_{ij})_{n \times n}$, $C = A \times B$

$$\begin{pmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,n-1} \\ c_{1,0} & c_{1,1} & & c_{1,n-1} \\ \vdots & \vdots & & \vdots \\ c_{n-1,0} & c_{n-1,1} & \cdots & c_{n-1,n-1} \end{pmatrix} = \begin{pmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n-1} \\ a_{1,0} & a_{1,1} & & a_{1,n-1} \\ \vdots & \vdots & & \vdots \\ a_{n-1,0} & a_{n-1,1} & \cdots & a_{n-1,n-1} \end{pmatrix} \cdot \begin{pmatrix} b_{0,0} & b_{0,1} & \cdots & b_{0,n-1} \\ b_{1,0} & b_{1,1} & & b_{1,n-1} \\ \vdots & \vdots & & \vdots \\ b_{n-1,0} & b_{n-1,1} & \cdots & b_{n-1,n-1} \end{pmatrix}$$



$$c_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$$

A中元素的第1下标与B中元素的第2下标相一致（对准）。

块棋盘划分

- A, B, C 分成 $p = \sqrt{p} \times \sqrt{p}$ 的方块阵 A_{ij}, B_{ij}, C_{ij} , 大小均为 $\frac{n}{\sqrt{p}} \times \frac{n}{\sqrt{p}}$ 。
 - p 个处理器编号为 $P_{0,0}, \dots, P_{0,\sqrt{p}-1}, \dots, P_{\sqrt{p}-1,\sqrt{p}-1}$ 。
 - P_{ij} 存放 A_{ij}, B_{ij}, C_{ij} 。
 - $C_{ij} = \sum_{k=0}^{\sqrt{p}-1} A_{ik} \cdot B_{kj}$ 。

$A_{0,0}$ $B_{0,0}$	$A_{0,1}$ $B_{0,1}$	$A_{0,2}$ $B_{0,2}$	$A_{0,3}$ $B_{0,3}$
$A_{1,0}$ $B_{1,0}$	$A_{1,1}$ $B_{1,1}$	$A_{1,2}$ $B_{1,2}$	$A_{1,3}$ $B_{1,3}$
$A_{2,0}$ $B_{2,0}$	$A_{2,1}$ $B_{2,1}$	$A_{2,2}$ $B_{2,2}$	$A_{2,3}$ $B_{2,3}$
$A_{3,0}$ $B_{3,0}$	$A_{3,1}$ $B_{3,1}$	$A_{3,2}$ $B_{3,2}$	$A_{3,3}$ $B_{3,3}$

块棋盘划分的矩阵乘法

- 每行处理器进行 A 矩阵块的多到多播送（得到 $A_{i,k}$, $k=0\sim\sqrt{p}-1$ ）。
- 每列处理器进行 B 矩阵块的多到多播送（得到 $B_{k,j}$, $k=0\sim\sqrt{p}-1$ ）。
- $P_{i,j}$ 执行计算： $C_{ij} = \sum_{k=0}^{\sqrt{p}-1} A_{ik} \cdot B_{kj}$ 。
- 缺点是存储较大。
 - ▶ 每个处理器需存储 $2\sqrt{p}$ 个块，每块大小为 n^2/p 。

块棋盘划分的矩阵乘法

```
#include "mpi.h"
#include <math.h>
#include <iostream>
using namespace std;
int main(int argc, char* argv[])
{
    int n = 1680; //矩阵大小为n*n
    int m; //子块大小为m*m
    int rank, p, q;
    if (argc > 1)
        n = atoi(argv[1]);
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD,
        &rank);
    MPI_Comm_size(MPI_COMM_WORLD,
        &p);
    q = int(sqrt(double(p)));
    m = n / q;
    double* a = new double[m * m]; //a[i*m+j]
    //为子块第i行第j列元素
    double* b = new double[m * m];
    double* c = new double[m * m];
```

```
double* aa = new double[m * n]; //存储当前子块所在行的A子块
double* bb = new double[n * m]; //存储当前子块所在列的B子块
srand(rank);
for (int i = 0; i < m * m; i++)
{
    a[i] = rand() * 1.0 / RAND_MAX;
    b[i] = rand() * 1.0 / RAND_MAX;
    c[i] = 0;
}
double start = MPI_Wtime();
MPI_Comm rowComm, columnComm;
MPI_Comm_split(MPI_COMM_WORLD,
    rank / q, rank % q, &rowComm);
MPI_Comm_split(MPI_COMM_WORLD,
    rank % q, rank / q, &columnComm);
MPI_Allgather(a, m * m, MPI_DOUBLE,
    aa, m * m, MPI_DOUBLE, rowComm);
MPI_Allgather(b, m * m, MPI_DOUBLE,
    bb, m * m, MPI_DOUBLE, columnComm);
for (int x = 0; x < q; x++)
```

```
{
    int offset = x * m * m;
    for (int i = 0; i < m; i++)
        for (int j = 0; j < m; j++)
            for (int k = 0; k < m; k++)
                c[i * m + j] += aa[offset + i * m + k] *
                    bb[offset + k * m + j];
}
MPI_Barrier(MPI_COMM_WORLD);
double finish = MPI_Wtime();
if (rank == 0)
    cout << "Elapsed time = " << finish - start
        << endl;
delete[] a;
delete[] b;
delete[] c;
delete[] aa;
delete[] bb;
MPI_Finalize();
return 0;
}
```

MPI_Barrier

- 最基础的同步函数，核心作用是让指定通信域内的所有进程在此处“等待集合”，直到该通信域内每一个进程都执行到这行代码，才会继续往下运行。
- 并行程序的执行时间由最慢的进程决定（木桶效应），而非所有进程耗时的平均值。
- MPI_Barrier确保所有进程已计算完毕。



通信域

■ 预定义的通信域

- ▶ MPI_COMM_WORLD: 所有进程的集合。
- ▶ MPI_COMM_SELF: 只包含使用它的进程。

■ MPI_Comm_split

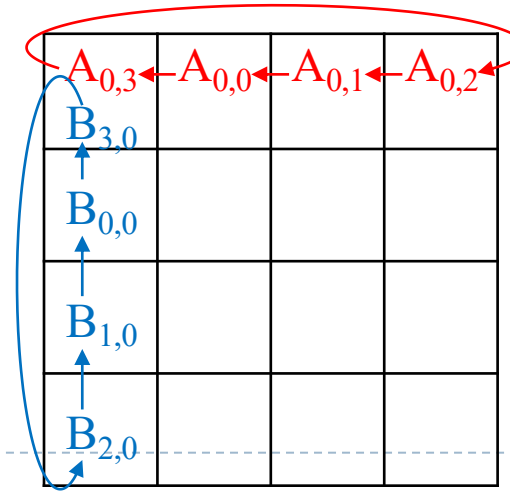
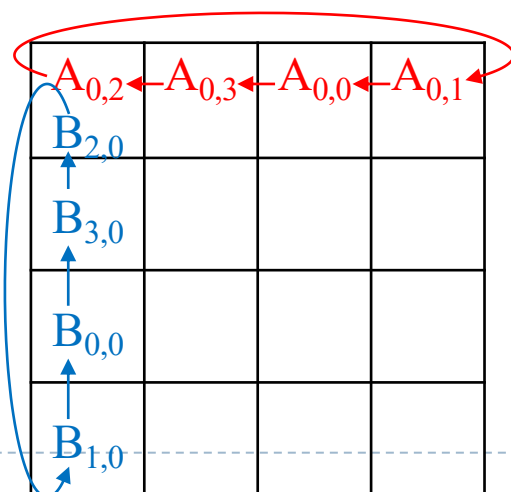
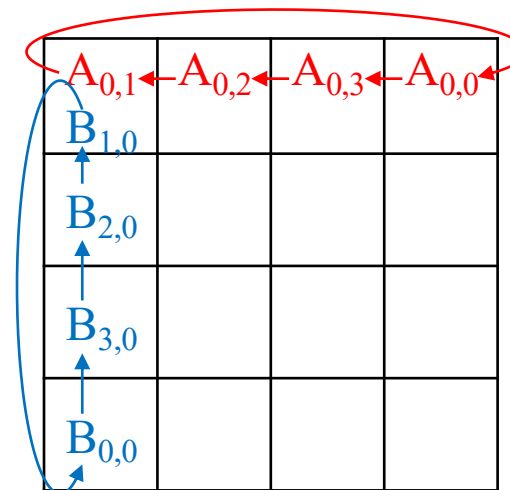
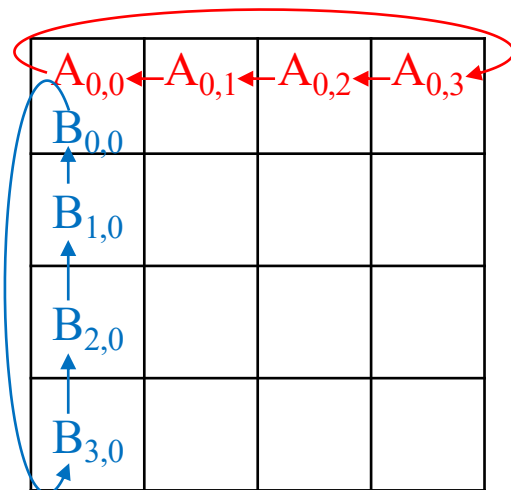
- ▶ 划分通信域。
- ▶ 进程之间通信时, 通信域要一致。

```
#include "mpi.h"
#include <iostream>
#include <cmath>
#include <windows.h>
using namespace std;
int main(void) {
    int rank, size;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int n = sqrt(size);
    MPI_Comm rowComm;
    MPI_Comm_split(MPI_COMM_WORLD, rank / n,
        rank % n, &rowComm);
    int rowRank, data = 0;
```

```
MPI_Comm_rank(rowComm, &rowRank);
if (rowRank == 0)
    data = rank * rank;
MPI_Bcast(&data, 1, MPI_INT, 0, rowComm);
Sleep(rank * 100);
cout << "P" << rank <<
    ":\t" << data;
fflush(stdout);
MPI_Finalize();
return 0;
}
```

Cannon乘法

$$C_{0,0} = A_{0,0}B_{0,0} + A_{0,1}B_{1,0} + A_{0,2}B_{2,0} + A_{0,3}B_{3,0}$$



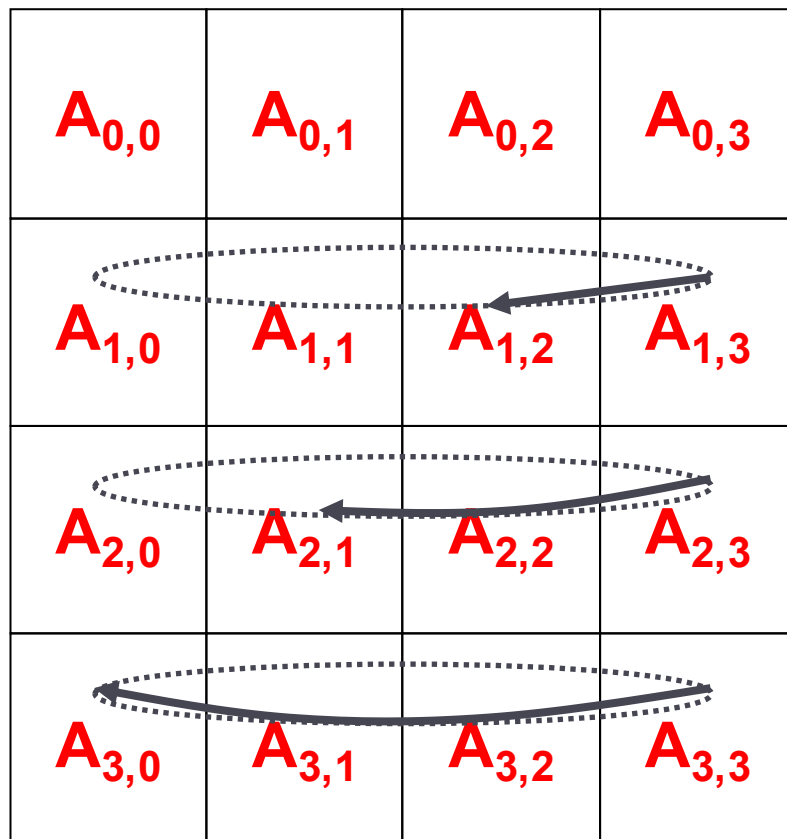
算法原理

- ① 所有块 $A_{i,j}$ ($0 \leq i, j \leq \sqrt{p}-1$) 向左循环移动 i 步（按行移位），
所有块 $B_{i,j}$ ($0 \leq i, j \leq \sqrt{p}-1$) 向上循环移动 j 步（按列移位）。
- ② 所有处理器 $P_{i,j}$ 做执行 $A_{i,j}$ 和 $B_{i,j}$ 的乘-加运算。
- ③ A 的每个块向左循环移动一步，
 B 的每个块向上循环移动一步。
- ④ 转②执行 $\sqrt{p}-1$ 次。

示例

Initial alignment of A

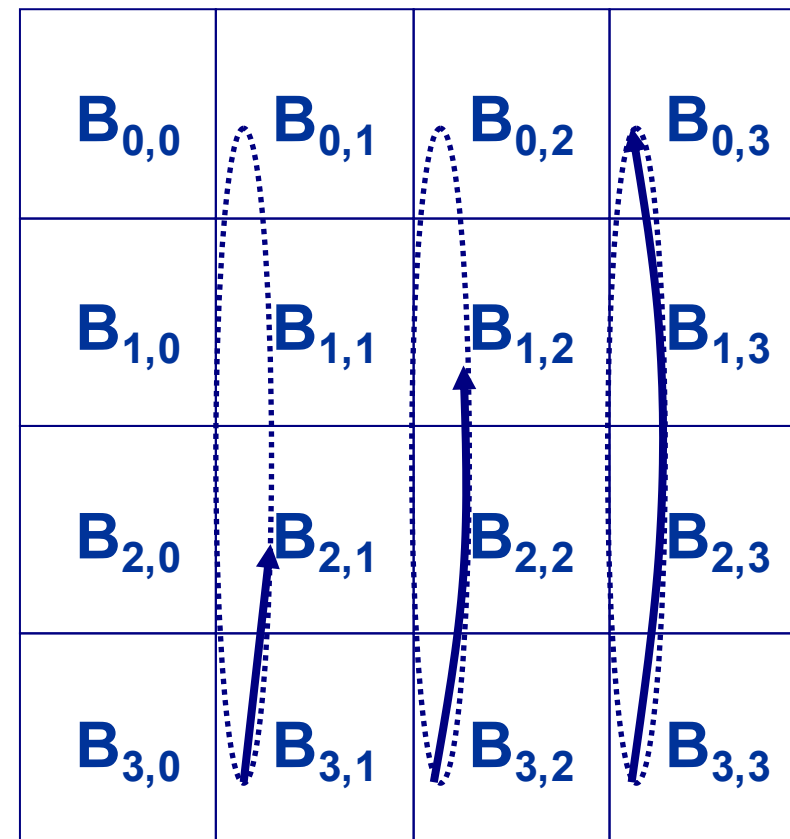
$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$
$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$
$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$
$A_{3,0}$	$A_{3,1}$	$A_{3,2}$	$A_{3,3}$



The diagram illustrates the initial alignment of matrix A, which is a 4x4 grid of elements labeled $A_{i,j}$ for $i, j \in \{0, 1, 2, 3\}$. The elements are arranged in four rows and four columns. Dashed arrows indicate the initial alignment path, which follows a row-wise traversal pattern: from $A_{0,0}$ to $A_{0,3}$, then down to $A_{1,0}$, across to $A_{1,3}$, down to $A_{2,0}$, across to $A_{2,3}$, and finally down to $A_{3,0}$ and across to $A_{3,3}$.

Initial alignment of B

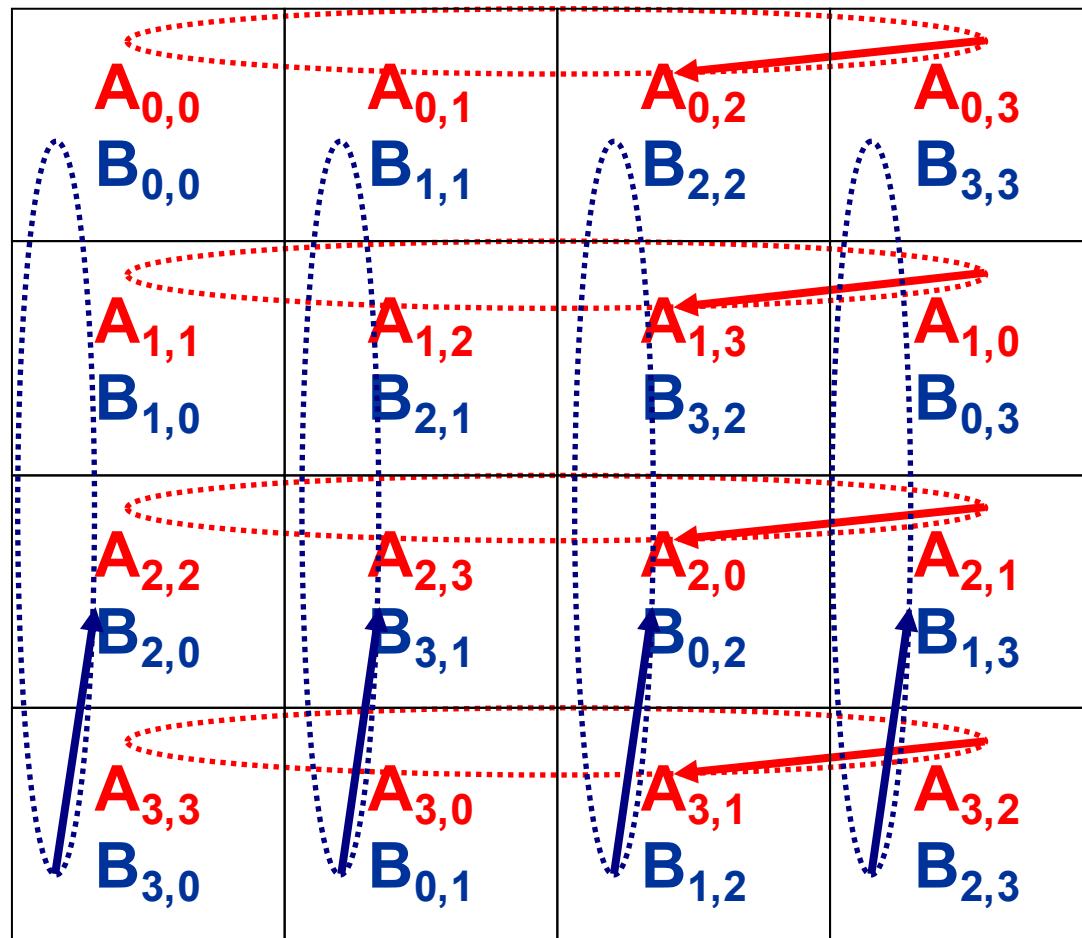
$B_{0,0}$	$B_{0,1}$	$B_{0,2}$	$B_{0,3}$
$B_{1,0}$	$B_{1,1}$	$B_{1,2}$	$B_{1,3}$
$B_{2,0}$	$B_{2,1}$	$B_{2,2}$	$B_{2,3}$
$B_{3,0}$	$B_{3,1}$	$B_{3,2}$	$B_{3,3}$



The diagram illustrates the initial alignment of matrix B, which is a 4x4 grid of elements labeled $B_{i,j}$ for $i, j \in \{0, 1, 2, 3\}$. The elements are arranged in four rows and four columns. Dashed arrows indicate the initial alignment path, which follows a column-wise traversal pattern: from $B_{0,0}$ down to $B_{3,0}$, then right to $B_{3,1}$, up to $B_{0,1}$, right to $B_{0,2}$, down to $B_{3,2}$, up to $B_{0,3}$, and finally down to $B_{3,3}$.

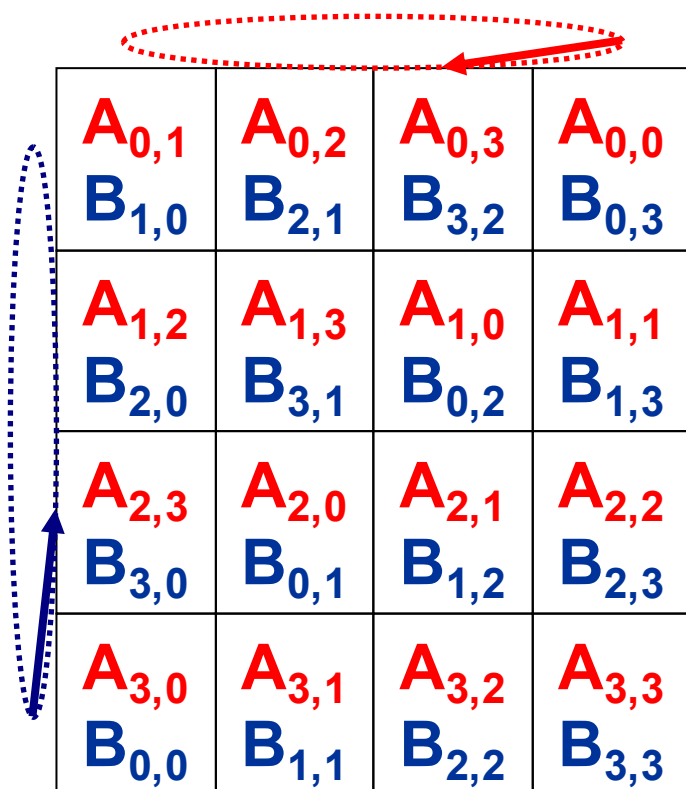
示例

A and B after initial alignment and shifts after every step



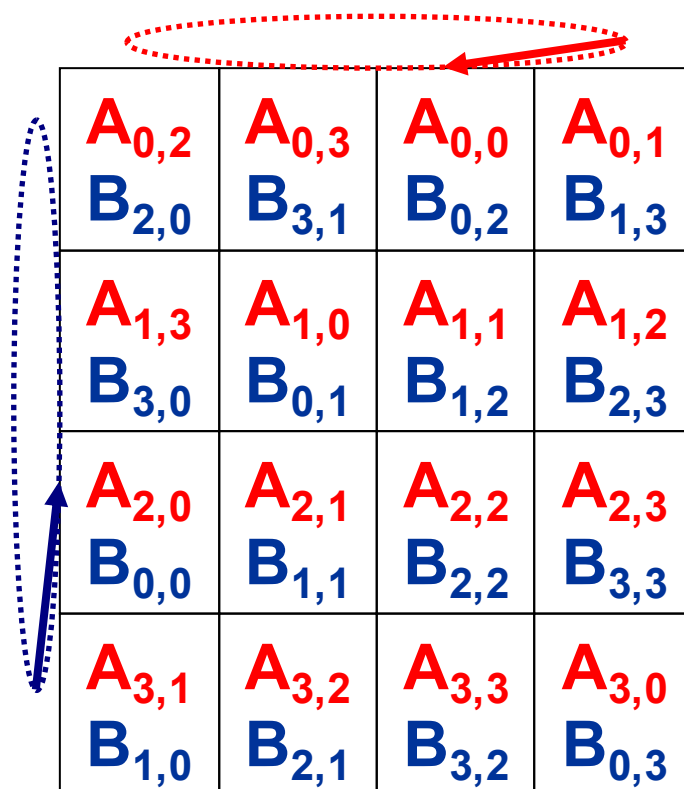
示例

After first shift



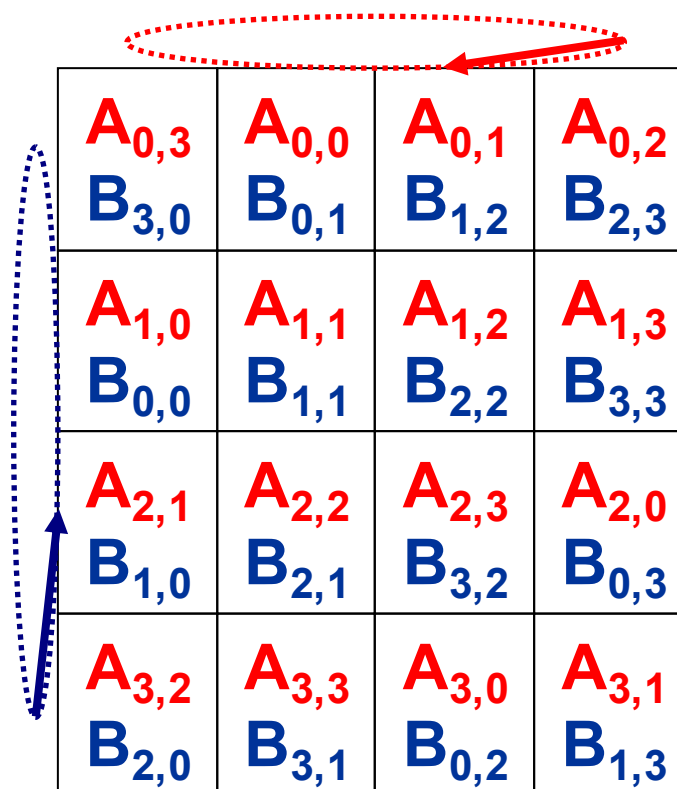
$A_{0,1}$ $B_{1,0}$	$A_{0,2}$ $B_{2,1}$	$A_{0,3}$ $B_{3,2}$	$A_{0,0}$ $B_{0,3}$
$A_{1,2}$ $B_{2,0}$	$A_{1,3}$ $B_{3,1}$	$A_{1,0}$ $B_{0,2}$	$A_{1,1}$ $B_{1,3}$
$A_{2,3}$ $B_{3,0}$	$A_{2,0}$ $B_{0,1}$	$A_{2,1}$ $B_{1,2}$	$A_{2,2}$ $B_{2,3}$
$A_{3,0}$ $B_{0,0}$	$A_{3,1}$ $B_{1,1}$	$A_{3,2}$ $B_{2,2}$	$A_{3,3}$ $B_{3,3}$

After second shift



$A_{0,2}$ $B_{2,0}$	$A_{0,3}$ $B_{3,1}$	$A_{0,0}$ $B_{0,2}$	$A_{0,1}$ $B_{1,3}$
$A_{1,3}$ $B_{3,0}$	$A_{1,0}$ $B_{0,1}$	$A_{1,1}$ $B_{1,2}$	$A_{1,2}$ $B_{2,3}$
$A_{2,0}$ $B_{0,0}$	$A_{2,1}$ $B_{1,1}$	$A_{2,2}$ $B_{2,2}$	$A_{2,3}$ $B_{3,3}$
$A_{3,1}$ $B_{1,0}$	$A_{3,2}$ $B_{2,1}$	$A_{3,3}$ $B_{3,2}$	$A_{3,0}$ $B_{0,3}$

After third shift



$A_{0,3}$ $B_{3,0}$	$A_{0,0}$ $B_{0,1}$	$A_{0,1}$ $B_{1,2}$	$A_{0,2}$ $B_{2,3}$
$A_{1,0}$ $B_{0,0}$	$A_{1,1}$ $B_{1,1}$	$A_{1,2}$ $B_{2,2}$	$A_{1,3}$ $B_{3,3}$
$A_{2,1}$ $B_{1,0}$	$A_{2,2}$ $B_{2,1}$	$A_{2,3}$ $B_{3,2}$	$A_{2,0}$ $B_{0,3}$
$A_{3,2}$ $B_{2,0}$	$A_{3,3}$ $B_{3,1}$	$A_{3,0}$ $B_{0,2}$	$A_{3,1}$ $B_{1,3}$

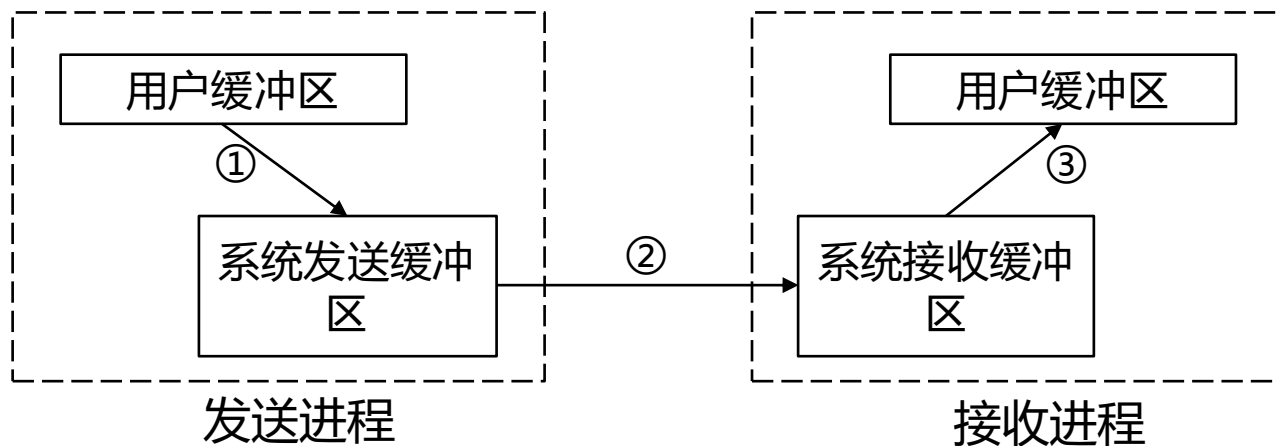
MPI_Send和MPI_Recv的语义

- 具体行为由MPI实现版本决定。
- MPI_Send的表现会因缓冲区大小、截断值和阻塞机制的不同而存在差异。
- MPI_Recv会始终处于阻塞状态，直至接收到匹配的消息。
- 务必了解你所使用的MPI实现版本，切勿主观臆断！



阻塞通信

- 调用返回时要求的通信操作已经正确完成。
 - ▶ 用户缓冲区可以使用。
- MPI_Send何时返回?
- MPI_Recv何时返回?

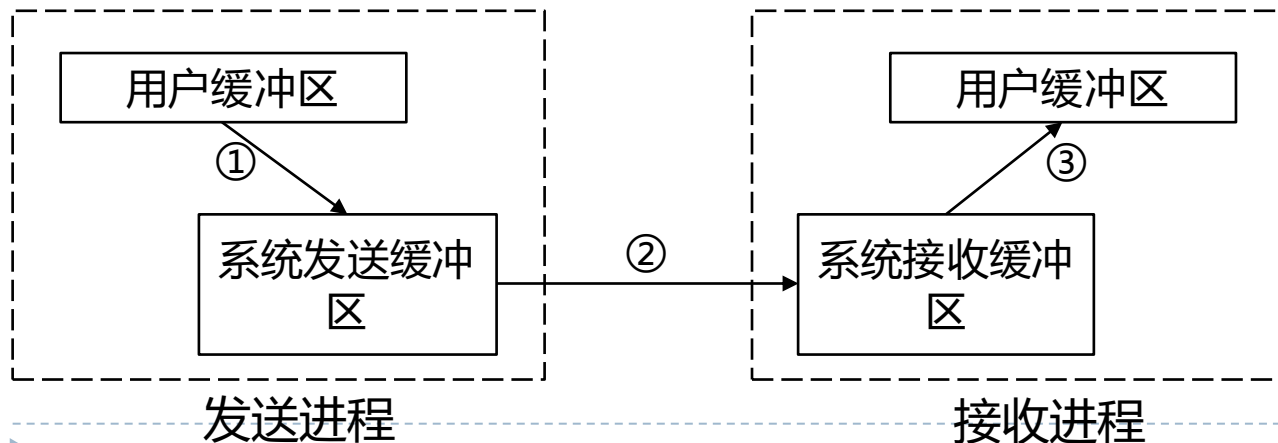


MPI_Send在①完成时返回
MPI_Recv在③完成时返回

```
#include "mpi.h"
#include <iostream>
#include <windows.h>
using namespace std;
const int n = 100; //10000
int main(void) {
    int* a = new int[n];
    int rank;
    MPI_Status status;
    MPI_Request request;
    MPI_Init(NULL, NULL);
    double t = MPI_Wtime();
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0) {
        for (int i = 0; i < n; i++)
            a[i] = i;
        MPI_Send(a, n, MPI_INT, 1, 0, MPI_COMM_WORLD);
        for (int i = 0; i < n; i++)
            a[i] = 0;
        cout << "time of send is "
    }
    else if (rank == 1) {
        Sleep(2000);
        MPI_Recv(a, n, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        cout << "time of recv is "
    }
    MPI_Finalize();
    return 0;
}
```

死锁

- 系统发送缓冲区容量受限。
- 当MPI_Recv调用时，才会建立通信，执行②。
- 避免死锁
 - ▶ 打破对称。
 - ▶ 使用非阻塞通信。



```
#include "mpi.h"
#include <stdio.h>
const int n = 100000;
int main(void) {
    int* a = new int[n];
    int* b = new int[n];
    int rank;
    MPI_Status status;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_
    COMM_WORLD,
    &rank);
    if (rank == 0) {
        for (int i = 0; i < n; i++)
            a[i] = i;
        MPI_Send(a, n,
        MPI_INT, 1, 0,
        MPI_COMM_WORLD);
        MPI_Recv(b, n,
        MPI_INT, 1, 0,
        MPI_COMM_WORLD,
        &status);
    }
    else if (rank == 1) {
        for (int i = 0; i < n; i++)
            b[i] = i * 2;
        MPI_Send(b, n,
        MPI_INT, 0, 0,
        MPI_COMM_WORLD);
        MPI_Recv(a, n,
        MPI_INT, 0, 0,
        MPI_COMM_WORLD,
        &status);
    }
    printf("P%d
    completed\n", rank);
    delete[] a;
    delete[] b;
    MPI_Finalize();
    return 0;
}
```

MPI_Sendrecv

- 这是一种替代我们自行调度通信的方式。
- 在单次调用中同时执行阻塞发送与阻塞接收。
- dest和source可以相同，也可以不同。
- 该方式尤其实用，因为MPI会自动调度通信，避免程序死锁或崩溃。

```
int MPI_Sendrecv(  
    void*      send_buf_p      /* in */,  
    int        send_buf_size   /* in */,  
    MPI_Datatype send_buf_type /* in */,  
    int        dest            /* in */,  
    int        send_tag        /* in */,  
    void*      recv_buf_p      /* out */,  
    int        recv_buf_size   /* in */,  
    MPI_Datatype recv_buf_type /* in */,  
    int        source          /* in */,  
    int        recv_tag        /* in */,  
    MPI_Comm   communicator    /* in */,  
    MPI_Status* status_p       /* in */);
```


非阻塞通信

- 不必等到通信操作完成便可以返回，通信操作可以交给底层的通信系统去完成。
 - ▶ MPI_Isend
 - ▶ MPI_Irecv
- 调用返回并不保证资源的可再用性。
- 通信完成检测方法：
 - ▶ MPI_Wait函数
 - ▶ MPI_Test函数
- 实现通信和计算的重叠。

MPI程序中的安全性

- MPI标准允许MPI_Send有两种不同的行为方式：
 - ▶ 它可以简单地将消息复制到MPI管理的缓冲区中并返回，
 - ▶ 或者可以阻塞，直到对MPI_Recv的匹配调用开始。
- 许多MPI实现设置了一个阈值，在这个阈值处系统从缓冲切换到阻塞。
 - ▶ 相对较小的消息会被MPI_Send缓冲，
 - ▶ 但对于较大的消息则会阻塞。

MPI程序中的安全性

- 如果每个进程都执行MPI_Send，那么没有进程能够开始执行对MPI_Recv的调用，程序将挂起或死锁。
- 每个进程均被阻塞，等待一个永远不会发生的事件。
- 依赖MPI提供的缓冲的程序被认为是不安全的。
- 对于某个输入集，这样的程序运行起来可能没有问题，但是对于其他的输入集，它可能挂起或崩溃。

MPI_Send

- MPI标准定义的MPI_Send的替代方法。
- 额外的"s"代表同步，并且MPI_Send保证阻塞，直到匹配的接收开始。

```
int MPI_Send(  
    void*          msg_buf_p      /* in */,  
    int           msg_size       /* in */,  
    MPI_Datatype   msg_type      /* in */,  
    int           dest           /* in */,  
    int           tag            /* in */,  
    MPI_Comm       communicator  /* in */);
```

线性方程组的求解

线性方程组的求解

- 一个 n 个变量 $x_1, x_2, \dots, x_n$ 的**线性方程**可表示为 $a_1x_1 + a_2x_2 + \dots + a_nx_n = b$, 其中, $a_1, a_2, \dots, a_n$ 和 b 均为常数。
- 变量 $x_1, x_2, \dots, x_n$ 的一组线性方程称为 **n 元线性方程组**, 也详称为 **n 个变量 n 个方程线性联立方程组**, 简称为**线性系**, 可表示为:

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n &= b_1 \\a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n &= b_2 \\&\dots \\a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n &= b_n\end{aligned}$$

$$\begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}$$

$$\mathbf{Ax} = \mathbf{b}$$

线性方程组的求解

- 矩阵 **A 中非零元素的位置和值**决定了求解 x 的困难复杂程度。通常一个顺序求解 $Ax=b$ 的算法的时间复杂度为 $O(n^3)$ 。
- 一个 $n \times n$ 的矩阵是上三角的，如果 $i > j$ 时， $a_{ij}=0$ 。
- 一个 $n \times n$ 的矩阵是下三角的，如果 $i < j$ 时， $a_{ij}=0$ 。
- 一个 $n \times n$ 的矩阵是三对角的，当且仅当 $|i-j| > 1$ 时， $a_{ij}=0$ 。
- 在 $Ax=b$ 中，若系数矩阵 A 是上（下）三角的，则称其为上（下）三角线性方程组；若系数矩阵 A 是三对角的，则称其为三对角线性方程组。

上三角方程组的求解

$$\begin{array}{rcl} x_1 + x_2 - x_3 + 4x_4 & = & 8 \\ -2x_2 - 3x_3 + x_4 & = & 5 \\ 2x_3 - 3x_4 & = & 0 \\ 2x_4 & = & 4 \end{array}$$

$$\begin{array}{rcl} x_1 + x_2 - x_3 & = & 0 \\ -2x_2 - 3x_3 & = & 3 \\ 2x_3 & = & 6 \\ 2x_4 & = & 4 \end{array}$$

(1) for $i=n$ to 1 do

(2) $x_i = b_i / a_{ii}$

(3) for $j=1$ to $i-1$ do

$$b_j = b_j - a_{ji}x_i$$

$$\begin{array}{rcl} x_1 + x_2 & = & 3 \\ -2x_2 & = & 12 \\ 2x_3 & = & 6 \\ 2x_4 & = & 4 \end{array}$$

$$\begin{array}{rcl} x_1 & = & 9 \\ -2x_2 & = & 12 \\ 2x_3 & = & 6 \\ 2x_4 & = & 4 \end{array}$$

三对角方程组直接求解法

■ 三对角方程组的一般形式

$$\begin{aligned}g_1x_1+h_1x_2&=b_1\\f_ix_{i-1}+g_ix_i+h_ix_{i+1}&=b_i, \quad 2\leq i\leq n-1\\f_nx_{n-1}+g_nx_n&=b_n\end{aligned}$$

(1) for $i=1$ to $n-1$ do

$$g_{i+1}=g_{i+1}-(f_{i+1}/g_i)h_i$$

$$b_{i+1}=b_{i+1}-(f_{i+1}/g_i)b_i$$

(2) for $i=n$ to 2 do

$$x_i=b_i/g_i$$

$$b_{i-1}=b_{i-1}-x_ih_{i-1}$$

(3) $x_1=b_1/g_1$

$$16x_1+4x_2=8$$

$$4x_1+11x_2-5x_3=7$$

$$2x_2+14x_3-6x_4=13$$

$$5x_3+18x_4=4$$

$$16x_1+4x_2=8$$

$$10x_2-5x_3=5$$

$$2x_2+14x_3-6x_4=13$$

$$5x_3+18x_4=4$$

$$16x_1+4x_2=8$$

$$10x_2-5x_3=5$$

$$15x_3-6x_4=12$$

$$5x_3+18x_4=4$$

$$16x_1+4x_2=8$$

$$10x_2-5x_3=5$$

$$15x_3-6x_4=12$$

$$20x_4=20$$

三对角方程组奇偶归约求解法

$$4x_1 + x_2 = 2 \quad \textcircled{1}$$

$$4x_1 + 11x_2 - 5x_3 = 7 \quad \textcircled{2}$$

$$2x_2 + 14x_3 - 6x_4 = 13 \quad \textcircled{3}$$

$$5x_3 + 18x_4 - 2x_5 = 20 \quad \textcircled{4}$$

$$2x_4 - x_5 = 0 \quad \textcircled{5}$$

$$\textcircled{1} - \frac{1}{11}\textcircled{2}: \quad 8x_1 + x_3 = 3 \quad \textcircled{6}$$

$$-\frac{2}{11}\textcircled{2} + \textcircled{3} + \frac{1}{3}\textcircled{4}: \quad -24x_1 + 547x_3 - 22x_5 = 607 \quad \textcircled{7}$$

$$-\frac{1}{9}\textcircled{4} + \textcircled{5}: \quad -5x_3 - 7x_5 = -20 \quad \textcircled{8}$$

$$-\textcircled{1} + \textcircled{2} + \frac{5}{14}\textcircled{3}: \quad 10x_2 - 2x_4 = 9 \quad \textcircled{9}$$

$$-\frac{5}{14}\textcircled{3} + \textcircled{4} - 2\textcircled{5}: \quad -10x_2 + 226x_4 = 215 \quad \textcircled{10}$$

$$\textcircled{6} - \frac{1}{547}\textcircled{7}: \quad 200x_1 + x_5 = 47 \quad \textcircled{11}$$

$$3\textcircled{6} + \textcircled{7} - \frac{22}{7}\textcircled{8}: \quad 5x_3 = 6 \quad \textcircled{13}$$

$$\textcircled{9} + \frac{1}{113}\textcircled{10}: \quad 10x_2 = 11 \quad \textcircled{14} \quad \textcircled{9} + \textcircled{10}: \quad x_4 = 1 \quad \textcircled{15}$$

$$\frac{5}{547}\textcircled{7} + \textcircled{8}: \quad -40x_1 - 1313x_5 = -2635 \quad \textcircled{12}$$

$$\textcircled{11} + \frac{1}{1313}\textcircled{12}: \quad 40x_1 = 9 \quad \textcircled{16}$$

$$\frac{1}{5}\textcircled{11} + \textcircled{12}: \quad x_5 = 2 \quad \textcircled{17}$$

奇偶归约法原理

- 三对角方程可以写成如下形式:

$$f_i x_{i-1} + g_i x_i + h_i x_{i+1} = b_i, \quad 1 \leq i \leq n, f_1 = h_n = 0$$

- 将原三对角方程组分解成了仅含奇下标变量的子方程组和仅含偶下标变量的子方程组:

$$f_{i-1} x_{i-2} + g_{i-1} x_{i-1} + h_{i-1} x_i = b_{i-1} \quad (i-1)$$

$$f_i x_{i-1} + g_i x_i + h_i x_{i+1} = b_i \quad (i)$$

$$f_{i+1} x_i + g_{i+1} x_{i+1} + h_{i+1} x_{i+2} = b_{i+1} \quad (i+1)$$

$$-\frac{f_i}{g_{i-1}}(i-1) + (i) - \frac{h_i}{g_{i+1}}(i+1), \quad \text{可得:}$$

$$-\frac{f_i f_{i-1}}{g_{i-1}} x_{i-2} + \left(-\frac{f_i h_{i-1}}{g_{i-1}} + g_i - \frac{h_i f_{i+1}}{g_{i+1}} \right) x_i - \frac{h_i h_{i+1}}{g_{i+1}} x_{i+2} = -\frac{f_i b_{i-1}}{g_{i-1}} + b_i - \frac{h_i b_{i+1}}{g_{i+1}},$$

- 继续递归地运用在各自规模减半了的子方程组上。 $1 \leq i \leq n$

有回代的高斯消去法

- 求解过程分为消元和回代两个阶段，消元是将系数矩阵 A 化为上三角阵 T ，然后对 $Tx=c$ 进行回代求解。
- 消元过程中可以应用选主元方法，增加算法的数值稳定性。
- 处理器按行划分。

$$\begin{array}{rcl} x_1 & +x_2 & -x_3 +4x_4=8 \quad \textcircled{1} \\ 2x_1 & -2x_2 & -3x_3 +x_4=23 \quad \textcircled{2} \\ x_1 & +3x_2 & +2x_3 -3x_4=-9 \quad \textcircled{3} \\ x_1 & +2x_2 & +x_3 +2x_4=4 \quad \textcircled{4} \end{array}$$

$$\begin{array}{rcl} \textcircled{1} - \frac{1}{2}\textcircled{2}: & 2x_2 & +\frac{1}{2}x_3 +\frac{7}{2}x_4=-\frac{7}{2} \quad \textcircled{1} \\ & 2x_1 & -2x_2 -3x_3 +x_4=23 \quad \textcircled{2} \\ \textcircled{3} - \frac{1}{2}\textcircled{2}: & 4x_2 & +\frac{7}{2}x_3 -\frac{7}{2}x_4=-\frac{41}{2} \quad \textcircled{3} \\ \textcircled{4} - \frac{1}{2}\textcircled{2}: & 3x_2 & +\frac{5}{2}x_3 +\frac{3}{2}x_4=-\frac{15}{2} \quad \textcircled{4} \end{array}$$

$$\begin{array}{rcl} \textcircled{1} - \frac{1}{2}\textcircled{3}: & -\frac{5}{4}x_3 & +\frac{21}{4}x_4=\frac{27}{4} \quad \textcircled{1} \\ & 2x_1 & -2x_2 -3x_3 +x_4=23 \quad \textcircled{2} \\ & 4x_2 & +\frac{7}{2}x_3 -\frac{7}{2}x_4=-\frac{41}{2} \quad \textcircled{3} \\ \textcircled{4} - \frac{3}{4}\textcircled{3}: & -\frac{1}{8}x_3 & +\frac{33}{8}x_4=\frac{63}{8} \quad \textcircled{4} \end{array}$$

$$\begin{array}{rcl} & -\frac{5}{4}x_3 & +\frac{21}{4}x_4=\frac{27}{4} \quad \textcircled{1} \\ & 2x_1 & -2x_2 -3x_3 +x_4=23 \quad \textcircled{2} \\ & 4x_2 & +\frac{7}{2}x_3 -\frac{7}{2}x_4=-\frac{41}{2} \quad \textcircled{3} \\ \textcircled{4} - \frac{1}{10}\textcircled{1}: & \frac{18}{5}x_4 & =\frac{36}{5} \quad \textcircled{4} \end{array}$$

无回代的高斯-约旦法

$$x_1 + x_2 - x_3 + 4x_4 = 8 \quad ①$$

$$2x_1 - 2x_2 - 3x_3 + x_4 = 23 \quad ②$$

$$x_1 + 3x_2 + 2x_3 - 3x_4 = -9 \quad ③$$

$$x_1 + 2x_2 + x_3 + 2x_4 = 4 \quad ④$$

for $j=1$ to n do

 for $i=1$ to n do

 for $k=j$ to $n+1$ do

 if $i \neq j$ then

$$a_{ik} = a_{ik} - (a_{ij}/a_{jj})a_{jk}$$

$$\begin{array}{l} ① - \frac{1}{2}②: \quad 2x_2 + \frac{1}{2}x_3 + \frac{7}{2}x_4 = -\frac{7}{2} \quad ① \\ \quad \quad \quad 2x_1 - 2x_2 - 3x_3 + x_4 = 23 \quad ② \end{array}$$

$$\begin{array}{l} ③ - \frac{1}{2}②: \quad \quad \quad 4x_2 + \frac{7}{2}x_3 - \frac{7}{2}x_4 = -\frac{41}{2} \quad ③ \\ ④ - \frac{1}{2}②: \quad \quad \quad 3x_2 + \frac{5}{2}x_3 + \frac{3}{2}x_4 = -\frac{15}{2} \quad ④ \end{array}$$

$$\begin{array}{l} ① - \frac{1}{2}③: \quad \quad \quad -\frac{5}{4}x_3 + \frac{21}{4}x_4 = \frac{27}{4} \quad ① \\ ② + \frac{1}{2}③: \quad 2x_1 \quad \quad -\frac{5}{4}x_3 - \frac{3}{4}x_4 = \frac{51}{4} \quad ② \\ \quad \quad \quad 4x_2 + \frac{7}{2}x_3 - \frac{7}{2}x_4 = -\frac{41}{2} \quad ③ \\ ④ - \frac{3}{4}③: \quad \quad \quad -\frac{1}{8}x_3 + \frac{33}{8}x_4 = \frac{63}{8} \quad ④ \end{array}$$

$$\begin{array}{l} \quad \quad \quad -\frac{5}{4}x_3 + \frac{21}{4}x_4 = \frac{27}{4} \quad ① \\ ② - ①: \quad 2x_1 \quad \quad \quad -6x_4 = 6 \quad ② \\ ③ + \frac{14}{5}①: \quad \quad 4x_2 \quad \quad + \frac{56}{5}x_4 = -\frac{8}{5} \quad ③ \\ ④ - \frac{1}{10}①: \quad \quad \quad \frac{18}{5}x_4 = -\frac{36}{5} \quad ④ \end{array}$$

$$\begin{array}{l} ① - \frac{35}{24}④: \quad \quad \quad -\frac{5}{4}x_3 = -\frac{15}{4} \quad ① \\ ② + \frac{5}{3}④: \quad 2x_1 \quad \quad \quad = 18 \quad ② \\ ③ - \frac{28}{9}④: \quad \quad 4x_2 \quad \quad \quad = -24 \quad ③ \\ \quad \quad \quad \frac{18}{5}x_4 = -\frac{36}{5} \quad ④ \end{array}$$

一种并行排序算法

排序

- n 个键和 $p=\text{size}$ 个进程。
- 为每个进程分配 n/p 个键。
- 没有限制将哪些键分配给哪些进程。
- 当算法终止时：
 - ▶ 分配给每个进程的键应该按照（假设）递增的顺序排序。
 - ▶ 如果 $0 \leq q < r < p$ ，那么分配给进程 q 的每个键 $\leq$ 分配给进程 r 的每个键。

进程	0	1	2	3
初始时	15, 11, 9, 16	3, 14, 8, 7	4, 6, 12, 10	5, 2, 13, 1
终止时	1, 2, 3, 4	5, 6, 7, 8	9, 10, 11, 12	13, 14, 15, 16

串行冒泡排序

```
void Bubble_sort(  
    int a[] /* in/out */,  
    int n /* in */) {  
    int list_length, i, temp;  
  
    for (list_length = n; list_length >= 2; list_length--)  
        for (i = 0; i < list_length - 1; i++)  
            if (a[i] > a[i+1]) {  
                temp = a[i];  
                a[i] = a[i+1];  
                a[i+1] = temp;  
            }  
  
} /* Bubble_sort */
```



奇偶移项排序

- 由两种不同类型的阶段序列组成。
- 在偶数阶段，比较交换：

$(a[0], a[1]), (a[2], a[3]), (a[4], a[5]), \dots$

- 在奇数阶段，比较交换：

$(a[1], a[2]), (a[3], a[4]), (a[5], a[6]), \dots$

示例

起始：5, 9, 4, 3。

阶段0：比较交换(5, 9)和(4, 3)，得到列表 5, 9, 3, 4。

阶段1：比较交换(9, 3)，得到列表 5, 3, 9, 4。

阶段2：比较交换(5, 3)和(9, 4)，得到列表 3, 5, 4, 9。

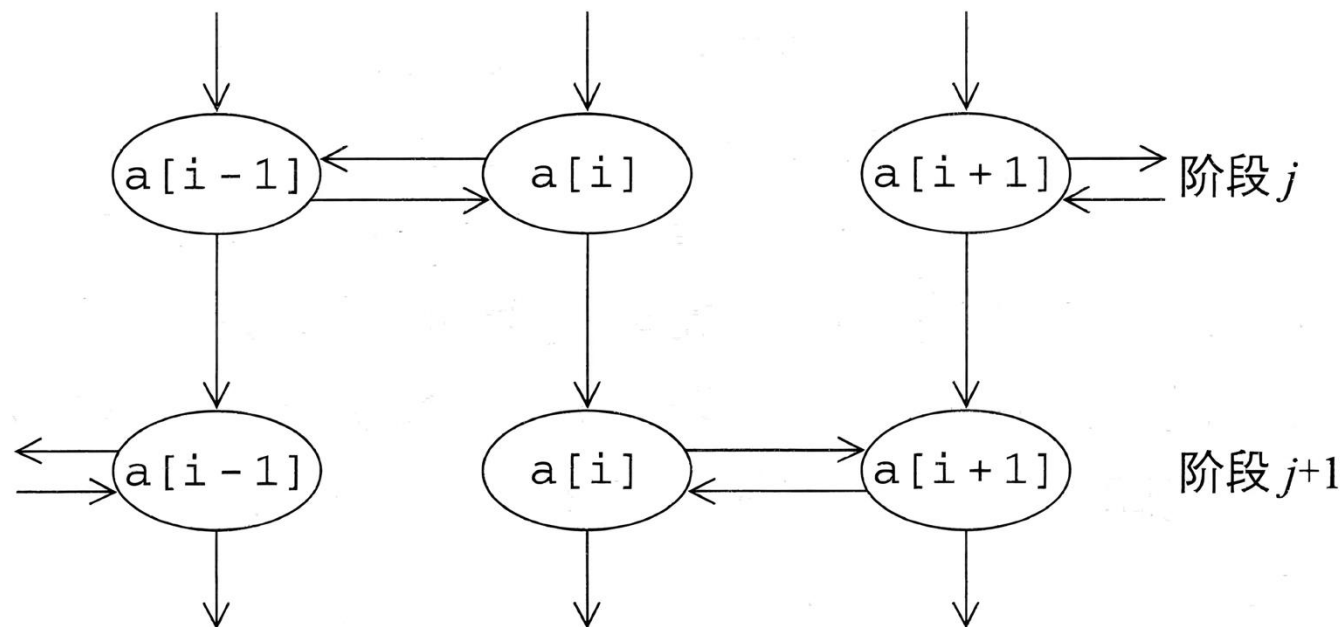
阶段3：比较交换(5, 4)，得到列表 3, 4, 5, 9。

串行奇偶移项排序

- 假设A是一个有 n 个键的列表，A是奇偶移项排序算法的输入。 n 个阶段之后，A会被排序。

```
void Odd_even_sort(  
    int a[] /* in/out */,  
    int n /* in */) {  
    int phase, i, temp;  
  
    for (phase = 0; phase < n; phase++)  
        if (phase % 2 == 0) { /* 偶数阶段 */  
            for (i = 1; i < n; i += 2)  
                if (a[i-1] > a[i]) {  
                    temp = a[i];  
                    a[i] = a[i-1];  
                    a[i-1] = temp;  
                }  
        } else { /* 奇数阶段 */  
            for (i = 1; i < n-1; i += 2)  
                if (a[i] > a[i+1]) {  
                    temp = a[i];  
                    a[i] = a[i+1];  
                    a[i+1] = temp;  
                }  
        }  
    }  
} /* Odd_even_sort */
```

奇偶排序中任务间的通信



确定 $a[k]$ 值的任务标记为 $a[k]$

并行奇偶移项排序

- 如果并行奇偶移项排序运行了 p 个进程，那么在 p 个阶段之后，输入列表将被排序。

时间	进程			
	0	1	2	3
开始	15, 11, 9, 16	3, 14, 8, 7	4, 6, 12, 10	5, 2, 13, 1
本地排序后	9, 11, 15, 16	3, 7, 8, 14	4, 6, 10, 12	1, 2, 5, 13
在阶段 0 后	3, 7, 8, 9	11, 14, 15, 16	1, 2, 4, 5	6, 10, 12, 13
在阶段 1 后	3, 7, 8, 9	1, 2, 4, 5	11, 14, 15, 16	6, 10, 12, 13
在阶段 2 后	1, 2, 3, 4	5, 7, 8, 9	6, 10, 11, 12	13, 14, 15, 16
在阶段 3 后	1, 2, 3, 4	5, 6, 7, 8	9, 10, 11, 12	13, 14, 15, 16

伪代码

对本地的键进行排序；

```
for (phase = 0; phase < comm_sz; phase++) {  
    partner = Compute_partner(phase, my_rank);  
    if (还未空闲) {  
        将键传给伙伴；  
        收到伙伴传来的键；  
        if (my_rank < partner)  
            保持更小的键；  
        else  
            保持更大的键；  
    }  
}
```

```
if (phase % 2 == 0)          /* 偶数阶段 */  
    if (my_rank % 2 != 0)     /* 奇数序列号 */  
        partner = my_rank - 1;  
    else                       /* 偶数序列号 */  
        partner = my_rank + 1;  
  
else                          /* 奇数阶段 */  
    if (my_rank % 2 != 0)     /* 奇数序列号 */  
        partner = my_rank + 1;  
    else                       /* 偶数序列号 */  
        partner = my_rank - 1;  
  
if (partner == -1 || partner == comm_sz)  
    partner = MPI_PROC_NULL;
```

并行奇偶移项排序中的Merge_low函数

```
void Merge_low(  
    int  my_keys[],      /* in/out   */  
    int  recv_keys[],   /* in      */  
    int  temp_keys[],   /* scratch */  
    int  local_n        /* = n/p, in */) {  
    int  m_i, r_i, t_i;  
  
    m_i = r_i = t_i = 0;  
    while (t_i < local_n) {  
        if (my_keys[m_i] <= recv_keys[r_i]) {  
            temp_keys[t_i] = my_keys[m_i];  
            t_i++; m_i++;  
        } else {  
            temp_keys[t_i] = recv_keys[r_i];  
            t_i++; r_i++;  
        }  
    }  
  
    for (m_i = 0; m_i < local_n; m_i++)  
        my_keys[m_i] = temp_keys[m_i];  
} /* Merge_low */
```

并行奇偶排序

```
#include <stdlib.h>
#include <iostream>
#include <algorithm>
#include <mpi.h>
using namespace std;
void Merge_low(int A[], int B[], int C[], int n) {
    int i, j, k;
    i = j = k = 0;
    while (k < n) {
        if (A[i] < B[j])
            C[k++] = A[i++];
        else
            C[k++] = B[j++];
    }
    void Merge_high(int A[], int B[], int C[], int n) {
        int i, j, k;
        i = j = k = n - 1;
        while (k >= 0) {
            if (A[i] > B[j])
                C[k--] = A[i--];
            else
                C[k--] = B[j--];
        }
    }
    void Odd_even_iter(int*& local_A, int partner_A[],
        int*& temp, int local_n, int phase, int rank, int p) {
```

```
    int partner = rank + (phase % 2 == rank % 2 ? 1 : -1);
    if (partner == -1 || partner == p)
        return;
    MPI_Sendrecv(local_A, local_n, MPI_INT, partner, 0,
        partner_A, local_n, MPI_INT, partner, 0,
        MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    if (rank < partner)
        Merge_low(local_A, partner_A, temp, local_n);
    else
        Merge_high(local_A, partner_A, temp, local_n);
    swap(local_A, temp);
}
int main(int argc, char* argv[]) {
    int rank, p, n = 32, local_n;
    double start, finish;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (argc > 1)
        n = atoi(argv[1]);
    local_n = n / p;
    int* local_A = new int[local_n];
    int* partner_A = new int[local_n];
    int* temp = new int[local_n];
    srand(rank);
    for (int i = 0; i < local_n; i++)
        local_A[i] = rand() % RAND_MAX;
```

```
    start = MPI_Wtime();
    sort(local_A, local_A + local_n);
    for (int phase = 0; phase < p; phase++)
        Odd_even_iter(local_A, partner_A, temp, local_n,
            phase, rank, p);
    MPI_Barrier(MPI_COMM_WORLD);
    finish = MPI_Wtime();
    if (rank == 0)
        cout << "Elapsed time = " << finish - start << endl;
    /*if (rank > 0)
        MPI_Recv(NULL, 0, MPI_INT, rank - 1, 0,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    cout << rank;
    for (int i = 0; i < local_n; i++)
        cout << "t" << local_A[i];
    cout << endl;
    if (rank < p - 1)
        MPI_Send(NULL, 0, MPI_INT, rank + 1, 0,
            MPI_COMM_WORLD);*/
    delete[] local_A;
    delete[] partner_A;
    delete[] temp;
    MPI_Finalize();
    return 0;
}
```


并行奇偶排序的运行时间

进程	键的数量 (千)				
	200	400	800	1 600	3 200
1	88	190	390	830	1 800
2	43	91	190	410	860
4	22	46	96	200	430
8	12	24	51	110	220
16	7.5	14	29	60	130

(时间以毫秒为单位)

小结

- MPI是一个函数库，可以从C或Fortran程序中调用。
- 通信域是可以互相发送消息的进程的集合。
- 许多并行程序使用单程序多数据（SPMD）方法。

小结

- 大多数串行程序是确定性的。这意味着如果我们用相同的输入运行相同的程序，将得到相同的输出。
- 并行程序通常不具有此属性。
- 集合通信涉及通信域中的所有进程。

小结

- 当我们对并行程序计时的時候，通常对消逝时间或“挂钟时间”感兴趣。
- 加速比是串行运行时间与并行运行时间的比率。
- 效率是加速比除以并行进程的数量。

小结

- 如果有可能增加问题的规模 n ，使效率不会随着 p 的增加而降低，那么并行程序就被称为可扩展的。
- 如果MPI程序的正确行为依赖于MPI_Send缓冲输入这一事实，那么这个MPI程序是不安全的。